

1. Patrones de diseño

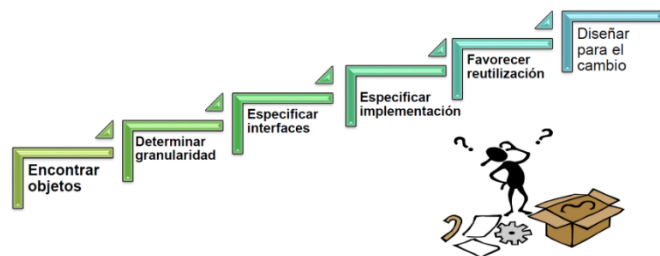
- **Definición:**

Los patrones de diseño son soluciones habituales a problemas que ocurren con frecuencia en el diseño de software. Es un concepto general para resolver un problema en particular, el cual debemos adaptar a nuestro código.

Estos patrones se aplican a nivel de código, es el nivel de patrón más bajo porque definen las **clases** y cómo se van a **relacionar** y que **métodos** se usarán para resolver un problema concreto de diseño. El problema mayormente se relaciona con **cohesión** o **acoplamiento**. La diferencia con los patrones arquitectónicos es estos operan a nivel de capas o módulos del sistema (mayor granularidad), mientras que los patrones de diseño trabajan con clases específicas y sus relaciones. Los patrones de diseño:

- ✓ Refinan un subsistema o componente de un sistema de software.
- ✓ Describen una estructura a la cual muchas veces recurrimos para resolver problemas de diseño.
- ✓ Son patrones de escala media.
- ✓ No afectan a la arquitectura de un sistema.
- ✓ No dependen del lenguaje de programación.
- ✓ Permiten resolver problemas complejos y direccionan la cooperación efectiva entre componentes.

- Aspectos que ayudan a resolver:



- a) **Encontrar Objetos** (en realidad lo que se encuentran son **clases**)

Se habla de encontrar clases, encontraremos clases de fabricación pura, las cuales no salen del dominio del problema y salen de la etapa de requerimientos.

Estos patrones nos ayudan a encontrar **clases de fabricación pura** (no tienen traza con el dominio del problema) para solucionar algún problema en particular. Se critica mucho a los patrones porque hacen crecer mucho al programa.

- b) **Determinar Granularidad**

Los patrones de diseño nos ayudan a tomar decisiones de cuántas responsabilidades asignar a una clase. Si una clase respeta el principio de **responsabilidad única**, el nivel de granularidad es fina, es decir, la clase va a tener muy pocos métodos que tienen que ver con el concepto que modela.

Entonces, determinar la granularidad de los objetos se basa en determinar cuántos métodos le voy a asignar a una clase o cómo voy a distribuir los métodos en las demás clases. Esto está determinado en cada patrón. Los objetos pueden variar enormemente de tamaño y número. Los patrones de diseño se encargan de decidir qué debería ser objeto porque un objeto puede representar cualquier cosa.

- c) **Especificar Interfaces**

Las interfaces son un tipo de clase abstracta que solo tiene la firma de los métodos, está la asignación del método que otra clase va a utilizar, no tienen código, son una declaración que va a utilizar otra clase. Se guía mucho con el polimorfismo, ya que cada clase que utilice a la interfaz va a implementar estos métodos de

forma diferente a otra clase, cuando hablamos de polimorfismo nos referimos a un método con la misma firma que va a reaccionar de forma diferente según la clase que lo utilice.

Si utilizamos relaciones de realización, las clases concretas implementan el comportamiento que está declarado en la interfaz. En el caso de usar herencia las clases concretas tienen la obligación de implementar todos los métodos del padre.

Se utiliza el principio SOLID de “*segregación de interfaces*”

La interfaz tiene sentido si tenemos múltiples implementaciones concretas que justifiquen la existencia de esta. Conviene usar interfaces para desacoplar al sistema.

d) Especificar implementación

Hay patrones que nos indican como deberíamos implementar el **código** dentro de nuestras clases. Por ejemplo, el singleton nos indica que debería contener el método new() de la clase de instancia única, así como que el getInstance() debería ser estático.

La implementación de un objeto queda definida por su clase, es decir un objeto se crea instanciando una clase. La clase especifica los datos, la representación interna del objeto y define las operaciones que puede realizar. Cuando se hace reuso de comportamiento/ de código el uso de la herencia queda justificado, pero para que la herencia esté bien diseñada, el hijo se tiene que comportar como el padre (liskov). Hay algunos patrones que sólo tienen sentido si se usan con herencia, si no pierden el propósito (template). Muchos patrones dependen de la distinción entre Herencia de clases y herencia de interfaces, recordando:

- Herencia de clases (generalización): define la implementación de un objeto en términos de la implementación de otro objeto. Es un mecanismo para compartir código y representación.
- Herencia de interfaces (realización): describe cuando se puede usar un objeto en lugar de otro.
 - ✓ Los patrones de creación permiten que un sistema esté basado en términos de interfaces y no en implementaciones.
 - ✓ Favorece reutilización → reuso de caja blanca (herencia) o de caja negra (delegación)
 - Delegación. → delega la solución del problema a la clase que lo resuelve o que tiene el código para hacerlo

e) Favorecer reutilización

Se habla de dos tipos de reuso de código:

- Reuso de caja blanca -> Herencia
- Reuso de caja negra -> Asociación (Agregación o composición)

El mecanismo que usamos en la caja negra es la **delegación**, es decir que le entrega el 100% de las responsabilidades a otro objeto para que lo haga por mí y no romper el encapsulamiento.

La clave para la **reutilización** es anticiparse a los nuevos requisitos y cambios en los mismos, de modo que los sistemas evolucionen de forma adecuada.

f) Diseñar para el Cambio

Nuestro software y las decisiones que tomemos tienen que estar pensados para que nuestro sistema pueda ser modificable e intercambiable, permitiendo evolucionar. Ni tan dura para que sea difícil ni tan blanda como para que sea muy fácil de cambiar.

Un diseño que no tenga en cuenta el **cambio** sufre el riesgo de tener que ser rediseñado por completo en el futuro. Los patrones de diseño ayudan a evitar esto al asegurar que un sistema pueda cambiar de forma concreta. (relación con evolución de software)

En la historia de los tres chanchitos, cada uno construye una casa: de paja, madera y ladrillos. Diseñar para el cambio con patrones de diseño es como ser el tercer chanchito, que se prepara para enfrentarse al lobo con una casa robusta y flexible ante amenazas. En lugar de construir algo rápidamente, como los otros dos chanchitos, utiliza estrategias que aseguren que la casa pueda resistir ataques (el cambio). Aunque requiere más esfuerzo inicial, esta preparación hace que la estructura sea adaptable y confiable, incluso cuando el entorno se vuelve impredecible.

- Tipos de patrones:

Los patrones de diseño son 23 y, se clasifican según su propósito:

- PATRONES DE CREACIÓN:** Son una categoría de patrones de diseño que se enfocan en cómo se crean los objetos, abstraen el proceso de instanciación y permiten desacoplar la creación de objetos del código que los utiliza. Esto facilita que los sistemas sean más flexibles y fáciles de modificar a medida que evolucionan.
- PATRONES DE ESTRUCTURA:** Explican cómo ensamblar objetos y clases en estructuras más grandes a la vez que mantiene la flexibilidad y eficiencia de la estructura.
- PATRONES DE COMPORTAMIENTO:** Se centran en la comunicación entre objetos, definiendo cómo interactúan y colaboran. Estos patrones ayudan a distribuir las responsabilidades entre los objetos y a manejar el flujo de control en un sistema, mejorando la flexibilidad y la reutilización del código.

		PROPÓSITO		
		Creación	Estructura	Comportamiento
ÁMBITO	Clase	Factory Method	Adapter (de clases)	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Adapter (de objetos) Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Clase: usa **herencia** (herencia de interfaz y de implementación) para variar la clase que es instanciada

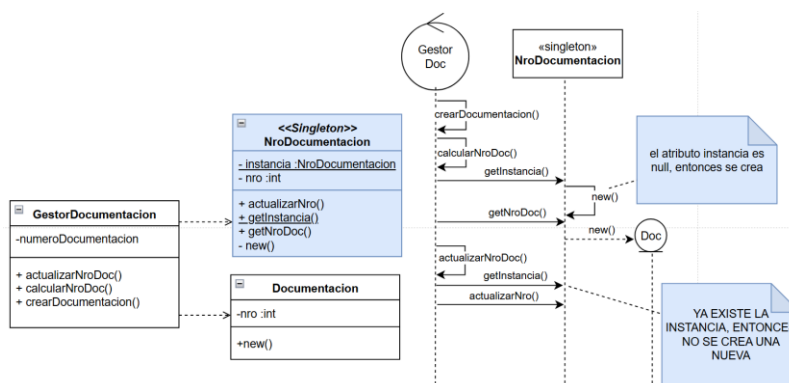
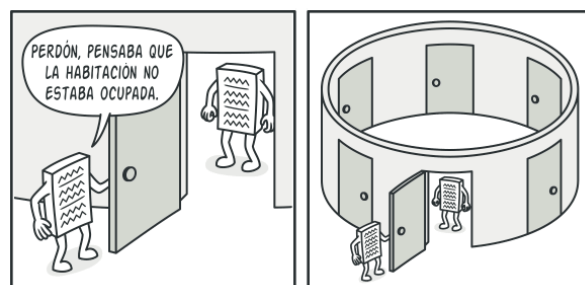
Objeto: usa **delegación** (agregación, composición, realización) para instanciar otro objeto

Singleton → Creación

Garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a esa instancia. Esto es útil cuando exactamente un objeto es necesario para coordinar acciones a través del sistema.

El motivo más habitual es controlar el acceso a algún recurso compartido, por ejemplo, una base de datos o un archivo.

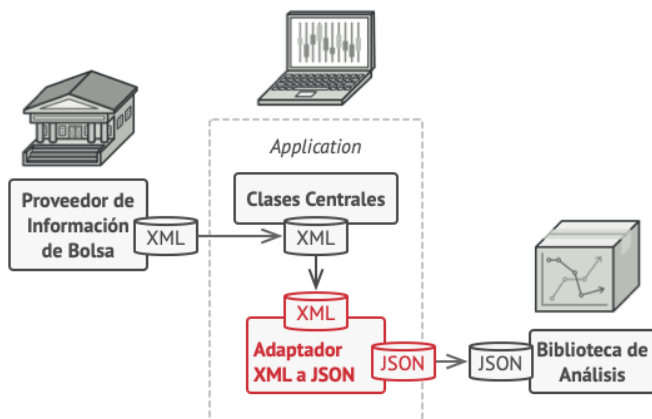
Ventaja: Permite un acceso controlado a la única instancia ya que posee un control estricto sobre cómo y cuándo acceden a ella.



En este ejemplo, el patrón **Singleton** garantiza que la clase **NroDocumentacion** tenga una única instancia durante la ejecución del programa. Esto se logra mediante el método **getInstancia()** (**ESTÁTICO**), que verifica si ya existe una instancia; si no es así, crea una nueva. La instancia única controla el acceso y actualización del atributo **nro**, que representa el número de documentación. Esto asegura que todas las partes del sistema (como **GestorDocumentacion**) utilicen el mismo número de documentación centralizado y actualizado, evitando inconsistencias y duplicaciones en la generación de números de documento.

Adapter → Estructura

El Patrón Adapter permite que dos interfaces incompatibles trabajen juntas. Actúa como un intermediario que convierte la interfaz de una clase en otra que el cliente espera, permitiendo que clases que no podrían trabajar juntas debido a incompatibilidades de interfaz se comuniquen entre sí.



En este ejemplo, el patrón Adapter se usa para convertir datos de un formato a otro. El proveedor de información de la bolsa entrega datos en **XML**, pero la **Biblioteca de Análisis** necesita esos mismos datos en **JSON** para hacer su trabajo. El **adaptador XML a JSON** toma los datos en XML, los convierte a JSON y luego los pasa a la biblioteca de análisis, lo que permite que dos sistemas que no son compatibles entre sí trabajen sin problemas. Este patrón ayuda a que los sistemas se comuniquen sin tener que cambiar su estructura interna.

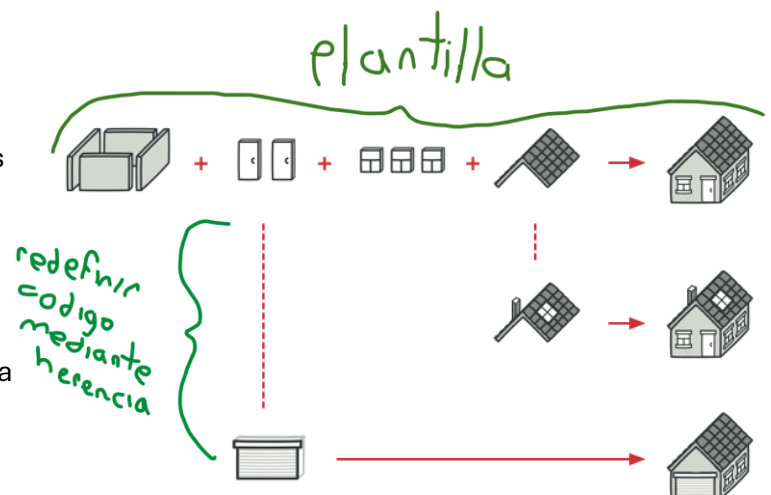
El Patrón Adapter es útil cuando:

1. Tienes clases existentes con interfaces que no coinciden con las que necesitas.
2. Quieres crear una interfaz común para varias clases que no son compatibles.
3. Deseas reutilizar código existente sin modificarlo.
4. Estás integrando sistemas o componentes que requieren adaptaciones para comunicarse correctamente

Template Method → Comportamiento

El Patrón Template Method define el esqueleto de un algoritmo en una operación, difiriendo algunos pasos a las subclases. Permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar la estructura de este. Se aplica **solo si hay herencia**.

El motivo principal para usar el Patrón Template Method es evitar la duplicación de código y permitir la reutilización de una estructura de algoritmo común, mientras se permite a las subclases proporcionar implementaciones específicas de ciertos pasos.



La **inversión de dependencia** en el patrón Template Method se logra al definir un método plantilla en una clase abstracta que delega ciertas partes del algoritmo a métodos abstractos implementados por subclases. Esto permite que el código de alto nivel no dependa de implementaciones concretas, mejorando la flexibilidad y mantenibilidad del sistema.

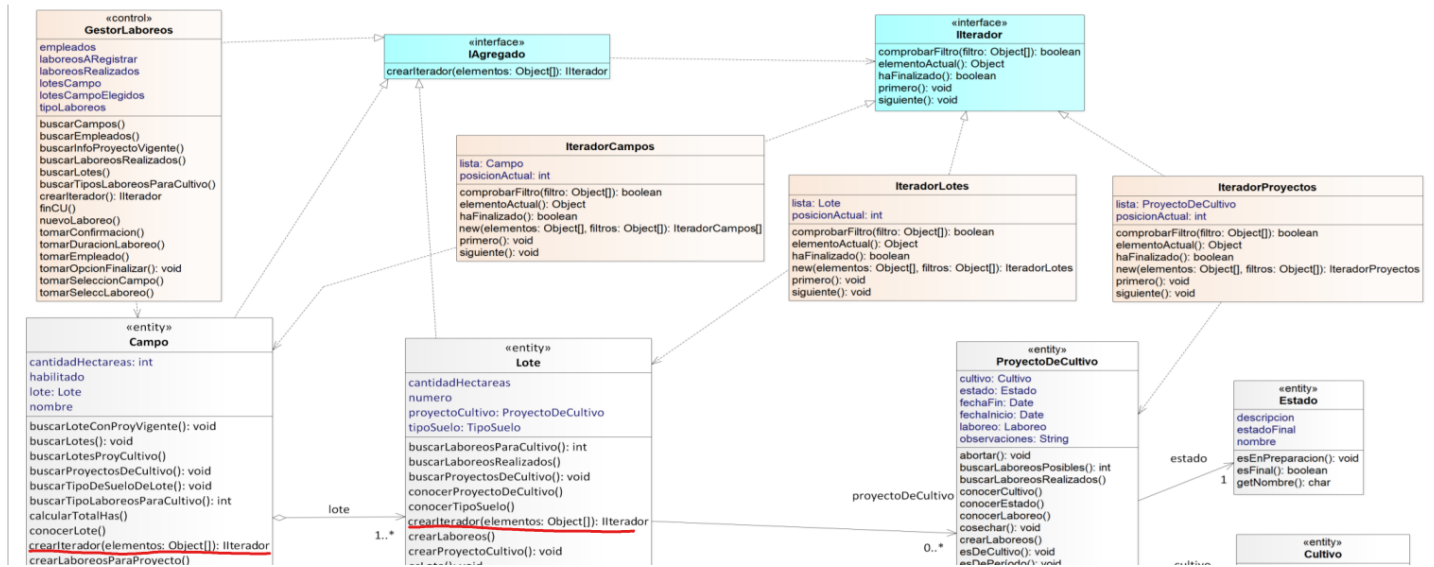
Iterator → Comportamiento

El Patrón Iterator proporciona una forma de acceder secuencialmente a los elementos de un objeto agregado (como una lista o una colección) sin exponer su representación subyacente. Permite recorrer colecciones de manera uniforme y abstraer el proceso de iteración, facilitando el acceso a los elementos de una colección sin preocuparse por su estructura interna.

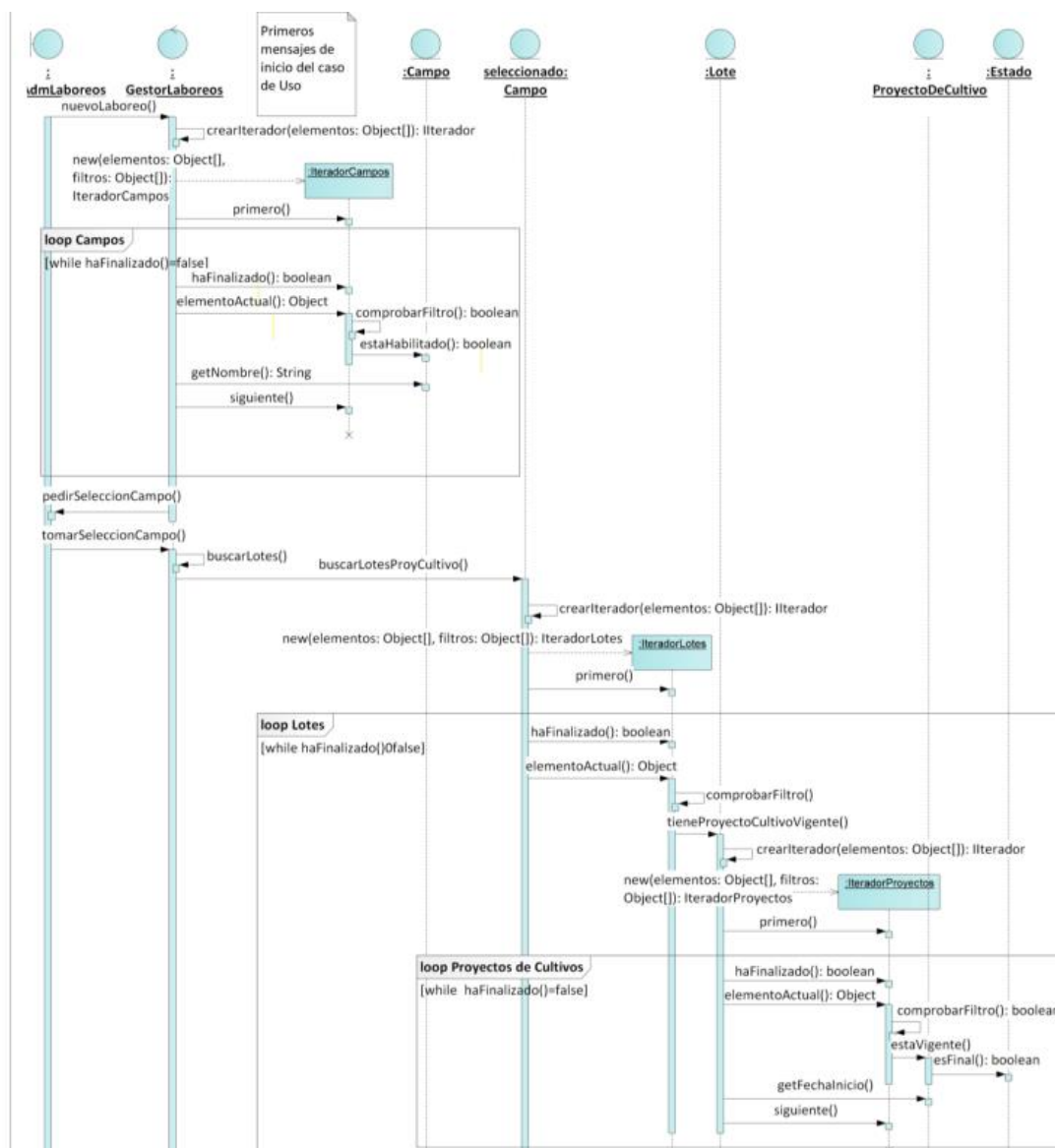
El motivo principal para usar el Patrón Iterator es separar la lógica de iteración de la colección en sí misma. Esto permite que diferentes tipos de colecciones puedan ser recorridas de manera uniforme mediante un iterador

El Patrón Iterator es útil cuando:

1. Quieres acceder a los elementos de una colección sin exponer su estructura interna.
2. Necesitas recorrer diferentes tipos de colecciones de manera uniforme.
3. Deseas proporcionar múltiples formas de iterar a través de una colección.
4. Quieres encapsular la lógica de iteración en un objeto separado para simplificar la interfaz de la colección.

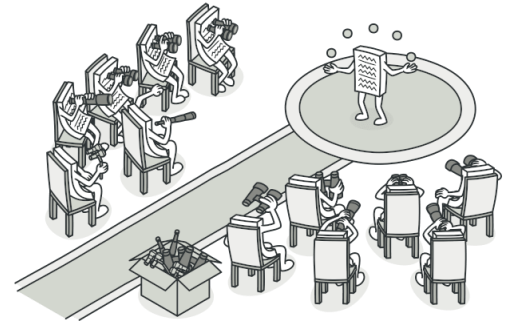


Como podemos ver en la vista estática, El iterador se encarga de iterar y buscar/comprobar lo que necesite, en vez de tener que hacerlo el gestor, delegando la responsabilidad de iteración. El iterador de campos itera sobre el de lotes y el de lotes itera sobre proyectos, por eso están los métodos `crearIterador` en las clases `Lote` y `Campo`. Se utiliza realización ya que se usan interfaces, donde se especifica el código en las clases que usan la interfaz "iterador"



Observer → Comportamiento

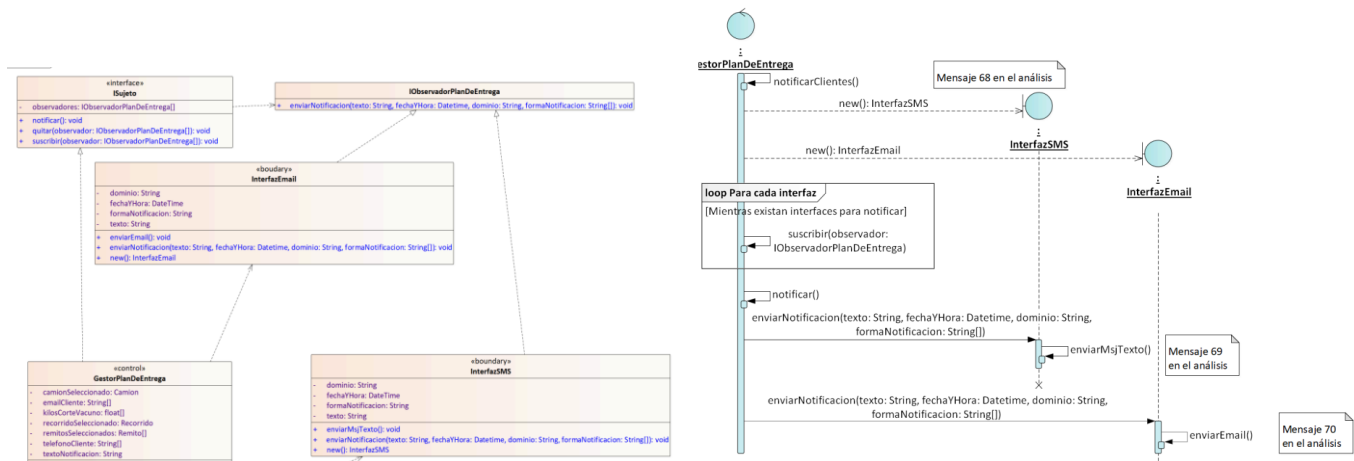
El Patrón Observer define una relación de dependencia uno a muchos entre objetos, de manera que cuando un objeto cambia de estado, todos sus dependientes son notificados y actualizados automáticamente. Este patrón es útil para implementar mecanismos de suscripción y notificación, permitiendo que un objeto emisor (sujeto) informe a múltiples objetos receptores (observadores) sobre cambios en su estado.



El motivo principal para usar el Patrón Observer es mantener la consistencia entre objetos relacionados sin acoplarlos estrechamente.

El Patrón Observer es útil cuando:

1. Un cambio en un objeto requiere cambios en otros objetos, y no se sabe cuántos objetos necesitan cambiar.
2. Quieres promover un bajo acoplamiento entre el sujeto y sus observadores.
3. Deseas que los objetos puedan suscribirse o cancelar su suscripción a eventos dinámicamente.
4. Estás diseñando sistemas donde un objeto debe notificar a otros objetos sobre cambios en su estado sin conocer sus detalles.



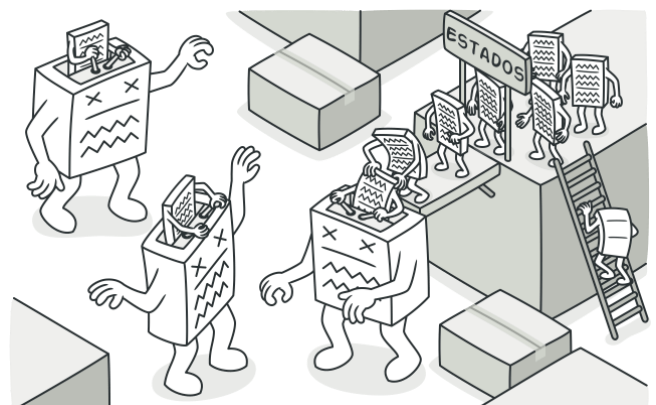
En este ejemplo, el patrón observer se usa para poder enviar notificaciones SMS y MAIL, agregando un método polimórfico “enviarNotificacion” a la interfaz observador que va a ser definido en las clases “hijo”. Como se ve en la dinámica, se suscribe a los observadores y luego se los notifica.

State → Comportamiento

El Patrón State permite que un objeto altere su comportamiento cuando su estado interno cambia. Este patrón se usa para encapsular los estados posibles de un objeto, delegando el comportamiento específico de cada estado a diferentes clases.

Es útil cuando un objeto debe cambiar su comportamiento dinámicamente en función de su estado actual. El motivo principal para usar el Patrón State es manejar cambios en el comportamiento de un objeto según su estado interno.

Esta estructura puede resultar similar al patrón Strategy, pero hay una diferencia clave. En el patrón State, los estados particulares pueden conocerse entre sí e iniciar transiciones de un estado a otro, mientras que las estrategias casi nunca se conocen.

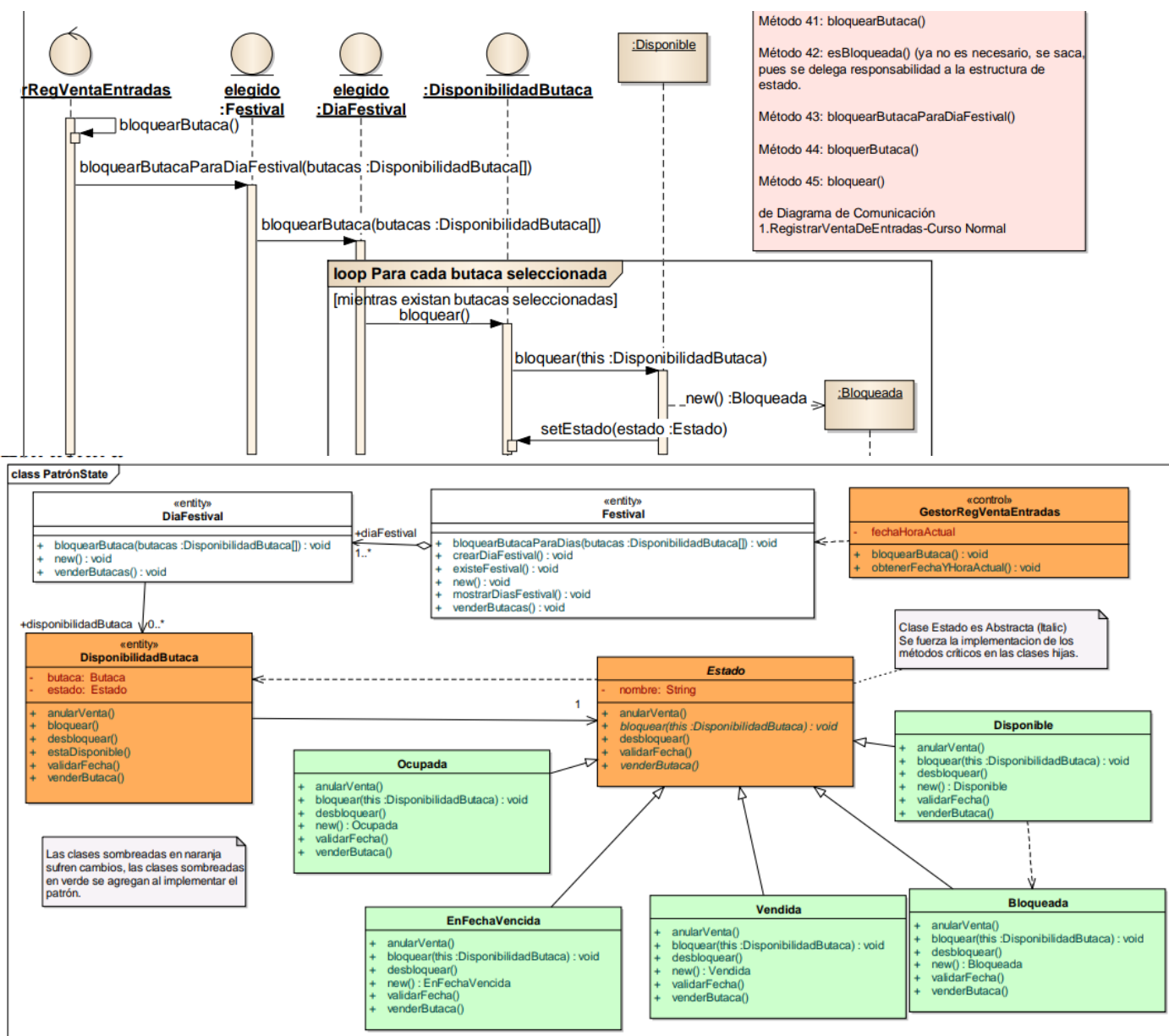


La imagen muestra la aplicación del patrón con personajes que representan diferentes estados y cómo cambian sus acciones. Este patrón facilita que el código sea más flexible y fácil de entender, ya que cada estado se maneja por separado, sin complicadas estructuras de condiciones.

La interfaz Estado declara los métodos específicos del estado. Estos métodos deben tener sentido para todos los estados concretos, porque no querrás que uno de tus estados tenga métodos inútiles que nunca son invocados. Los Estados Concretos proporcionan sus propias implementaciones para los métodos específicos del estado. Para evitar la duplicación de código similar a través de varios estados, pueden incluir clases abstractas intermedias que encapsulan algún comportamiento común. Los objetos de estado pueden almacenar una referencia inversa al objeto de contexto. A través de esta referencia, el estado puede extraer cualquier información requerida del objeto de contexto, así como iniciar transiciones de estado.

El Patrón State es útil cuando:

1. Necesitas que un objeto cambie su comportamiento dinámicamente según su estado.
2. Quieres evitar el uso de múltiples condicionales para manejar el comportamiento basado en estados.
3. Deseas hacer el código más mantenible y extensible al encapsular comportamientos específicos de cada estado en clases separadas.
4. Estás diseñando sistemas donde los estados y comportamientos pueden cambiar frecuentemente y necesitas una forma clara de manejar estos cambios.

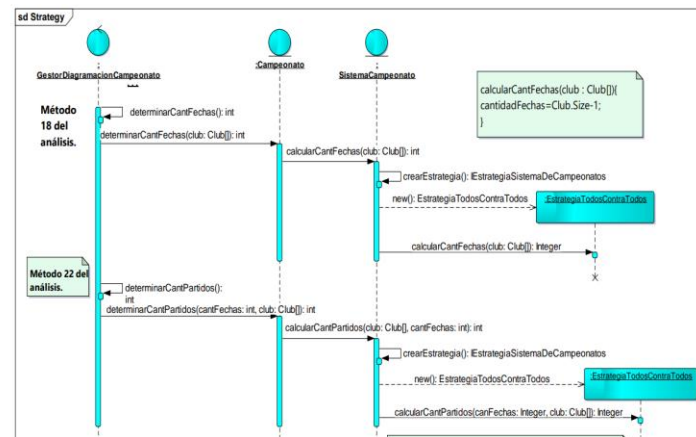
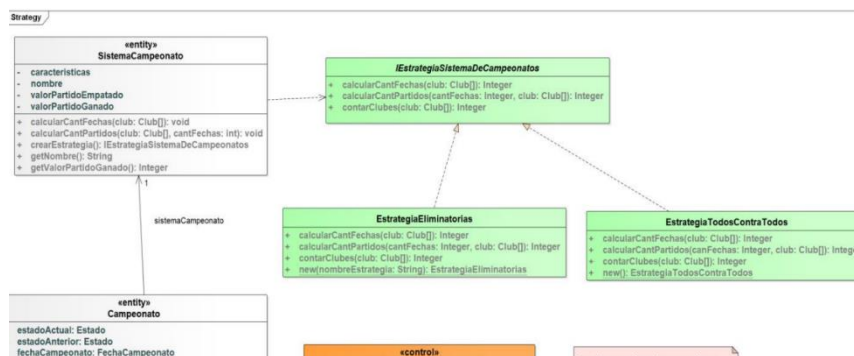


En este ejemplo, tenemos la clase CONTEXTO “DisponibilidadButaca” la cual tiene una clase abstracta “Estado”, la cual mediante herencia se relaciona con sus estados concretos, los cuales se encargarán de utilizar los métodos definidos en el padre, para instanciar nuevos cambios de estado. Se delega la creación de un nuevo estado al estado actual, en este caso el estado “Disponible” cambia el estado a “Bloqueada”

Strategy → Comportamiento

El Patrón Strategy permite definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables. Esto permite que los algoritmos varíen independientemente del cliente que los utiliza. Es útil cuando se desea que un objeto sea capaz de cambiar su comportamiento en tiempo de ejecución.

El motivo principal para usar el Patrón Strategy es evitar el uso de múltiples condicionales para seleccionar y ejecutar diferentes algoritmos.



En este ejemplo, el patrón strategy se encarga de crear dos clases Estrategia de fabricación pura, las cuales van a realizar los cálculos correspondientes de acuerdo con la Estrategia seleccionada, cada Estrategia define su propia forma de cálculo. Se utiliza realización

El Patrón Strategy es útil cuando:

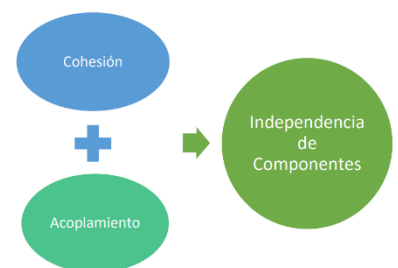
1. Necesitas que un objeto sea capaz de cambiar su comportamiento en tiempo de ejecución.
2. Quieres evitar el uso de múltiples condicionales (if-else o switch-case) para seleccionar algoritmos.
3. Deseas encapsular algoritmos en clases separadas para facilitar su mantenimiento y extensión.
4. Estás diseñando sistemas donde diferentes variantes de un algoritmo pueden aplicarse en diferentes contextos o condiciones

2. Principios de diseño (SOO y SOLID)

Un principio de diseño es una técnica o herramienta básica que puede ser aplicada para diseñar o construir software, para hacer más flexible, fácil de entender, de calidad, extensible y mantenible.

Diferencia con los Patrones de Diseño:

- **Patrones de diseño** son soluciones reutilizables a problemas comunes que se presentan en el desarrollo de software.
- Los **principios** no son soluciones directas a problemas concretos, sino que son **directrices** o **enfoques** que guían el diseño. Al seguir estos principios, el diseño se hace más limpio, escalable y mantenible, lo cual a largo plazo ayuda a evitar la creación de sistemas que son difíciles de modificar o que no pueden evolucionar con facilidad. Es decir, **los principios te ayudan a crear una base sólida para que los patrones de diseño sean más efectivos.**

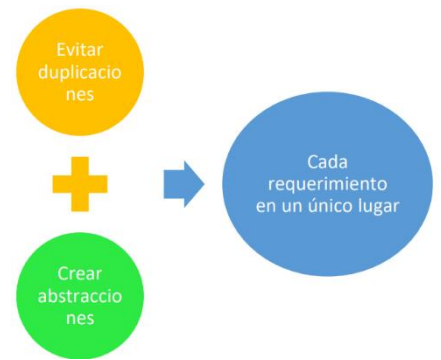


- **Lista de principios clave (importante saberlos bien):**

- 1. DRY – Don't Repeat Yourself (no te repitas):**

Promueve la eliminación de duplicación en el código, de modo que cada pieza de lógica o información esté definida en un único lugar. Esto es esencial para evitar problemas de mantenimiento y errores por inconsistencias.

Si tenemos una misma pieza de información ubicadas en dos lugares distintos (ej: dos clases que tienen el mismo método) y tenemos que cambiar la lógica de ese algoritmo, se tiene que hacer en todos los lugares en que aparezca y puede ocurrir que te olvidas de actualizarlo en uno de esos lugares y luego surgen problemas como inconsistencias o incoherencias.



- 2. TDA – Tell Don't Ask (pedir no preguntar)**



Establece que los objetos deben pedir a otros que hagan algo por ellos, en lugar de preguntarles por datos y tomar decisiones ellos mismos. Esencialmente, fomenta el **acoplamiento bajo** y la distribución de responsabilidades.

El patrón **Experto** está relacionado porque distribuye las responsabilidades hacia el objeto que tiene el conocimiento necesario para realizarlas.

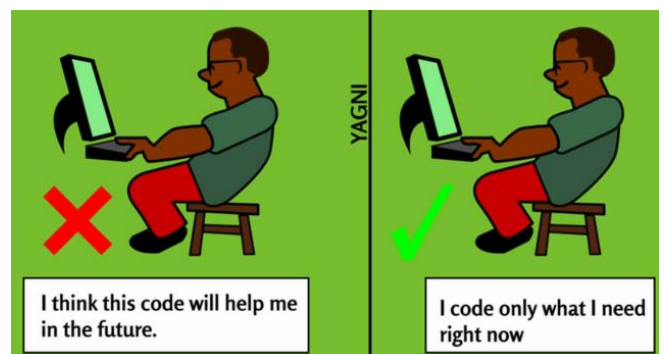
- 3. Principio KISS (keep it simple, stupid)**

Nos invita a mantener las cosas simples y claras en todo momento, tanto en el diseño como en el desarrollo de software. Este enfoque facilita la comprensión, el mantenimiento y la evolución de un sistema. Aplicar este principio significa evitar complejidades innecesarias, como agregar abstracciones que no tienen un propósito inmediato o introducir dependencias masivas cuando solo se necesitan unas pocas funcionalidades.

La descomposición de algo complejo en componentes simples es un enfoque arquitectónicamente correcto (aquí KISS se hace eco de DRY)

- 4. Principio YAGNI (you ain't gonna need it)**

Nos advierte contra la tentación de desarrollar funcionalidades adicionales pensando en posibles necesidades futuras. En lugar de anticiparse a problemas que quizás nunca surjan, YAGNI sugiere enfocarse únicamente en resolver los requisitos actuales. Esto no solo simplifica el diseño y el desarrollo, sino que también reduce la complejidad del código, facilitando su mantenimiento.

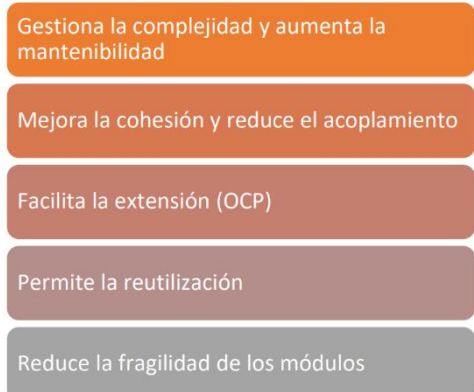


- 5. Separación de intereses (SoC)**

Propone dividir un sistema en partes, cada una encargada de una responsabilidad o preocupación específica. Esto no solo facilita la comprensión y el mantenimiento, sino que también permite modificar o extender partes del sistema sin afectar otra

La separación de intereses se puede aplicar principalmente en 2 niveles:

- Nivel arquitectónico: separar en varias capas.
- Nivel de programación: servicios se pueden separar en diferentes niveles.



Relación con principios SOLID:

Al aplicar la Separación de Intereses, se logra una arquitectura más organizada, y al seguir el SRP, te aseguras de que cada componente dentro de esas capas o módulos esté enfocado en una sola tarea, haciendo que el sistema sea más sencillo de entender y modificar

6. Principio de menor conocimiento / ley de demeter

DON'T TALK TO STRANGERS

Este principio propone que un objeto solo interactúe con objetos que conoce directamente y evita depender de cadenas largas de relaciones o estructuras internas de otros objetos. Esto fomenta un sistema menos acoplado, más modular y fácil de mantener.

Ventajas:

- Mayor legibilidad/cohesión, lo que aumenta la capacidad de mantenimiento y la adaptabilidad.
- Reduce el riesgo de que los métodos no revisados pasen desapercibidos en otra parte del código.
- Si los objetos están demasiado acoplados y el orden en que los métodos reciben los argumentos se altera, los argumentos deben cambiarse en cualquier otro lugar donde se llame al método dentro del código.

Desventajas:

- Requiere una gran cantidad de métodos de envoltorio (intermediarios) para transportar información entre objetos distantes, ya que la información no se puede recuperar de otra manera

7. Encapsular lo que varía (Open-Closed en SOLID)

Este principio se centra en identificar y aislar las partes del sistema que son susceptibles a cambios, separándolas de las partes que permanecen estables. Esto minimiza el impacto de los cambios en el resto del código, haciendo el sistema más mantenible y flexible.

Hay dos tipos de encapsulación

- A nivel de método

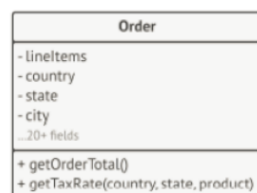
```
1 method getOrderTotal(order) is
2   total = 0
3   foreach item in order.lineItems
4     total += item.price * item.quantity
5
6   if (order.country == "US")
7     total += total * 0.07 // US sales tax
8   else if (order.country == "EU"):
9     total += total * 0.20 // European VAT
10
11  return total
```

ANTES: el código del cálculo del impuesto está mezclado con el resto del código.

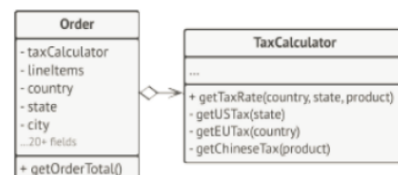
```
1 method getOrderTotal(order) is
2   total = 0
3   foreach item in order.lineItems
4     total += item.price * item.quantity
5
6   total += total * getTaxRate(order.country)
7
8   return total
9
10 method getTaxRate(country) is
11   if (country == "US"):
12     return 0.07 // US sales tax
13   else if (country == "EU"):
14     return 0.20 // European VAT
15   else
16     return 0
```

DESPUÉS: los cambios relacionados con el impuesto quedan aislados dentro de un único método.

- A nivel de clase



ANTES: el cálculo del impuesto está en la clase Pedido



DESPUÉS: el cálculo del impuesto se esconde dentro de la clase Pedido. Los objetos de la clase Pedido delegan todo el trabajo relacionado con el impuesto a un objeto especial.

Se relaciona directamente con el **principio SOLID de Abierto/Cerrado (Open/Close)**: el código debe estar abierto para la extensión (es decir, fácil de añadir nuevas funcionalidades) pero cerrado para la modificación (es decir, no modificar código existente innecesariamente).

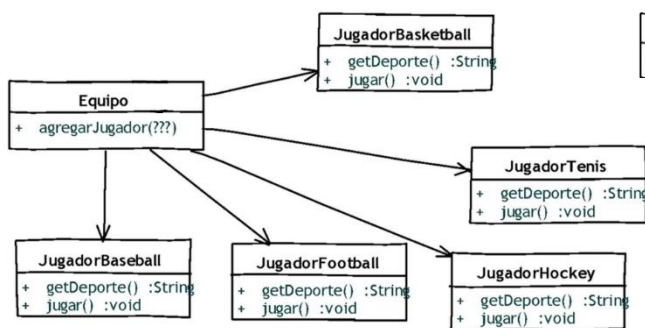
8. Programar hacia la interfaz y no una implementación

Este principio tiene como objetivo hacer que el código sea flexible y fácilmente extensible sin necesidad de modificar el código existente. Básicamente, significa que debes trabajar con **interfaz o clase abstracta** (abstracción) y no directamente con las **clases concretas** (implementaciones). Esto promueve una mejor **desacoplamiento** entre los componentes y reduce las dependencias de implementación entre los subsistemas.

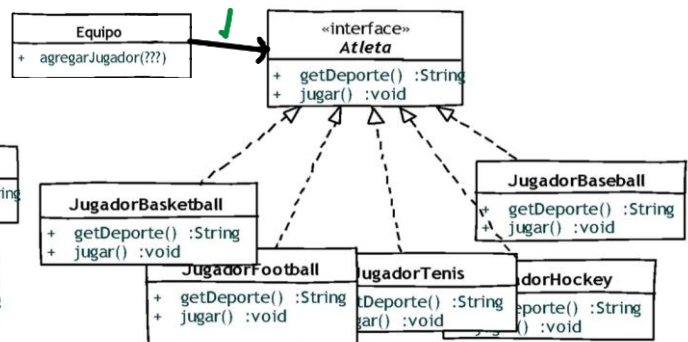
Imagina que tienes dos clases que necesitan colaborar, pero lo hacen de una manera muy acoplada porque una depende directamente de la otra. Esto no es muy flexible. El objetivo aquí es hacer que las clases dependan de **abstracciones** (interfaces o clases abstractas) en lugar de **concreciones** (implementaciones específicas).

Pasos para aplicar el principio:

1. **Determina las necesidades específicas:** Identifica qué métodos necesita un objeto de otro.
2. **Crea una interfaz o clase abstracta:** Define los métodos que el objeto va a necesitar en una interfaz o clase abstracta.
3. **Implementación de la interfaz:** Haz que las clases que antes estaban conectadas de manera rígida implementen esta interfaz.
4. **Usa la interfaz, no la clase concreta:** En lugar de hacer que una clase dependa de la implementación concreta, haz que dependa de la interfaz, lo que aumenta la flexibilidad.



Se programa hacia la implementación (clases concretas).
MUCHO ACOPLAMIENTO



Se programa hacia la interfaz "Atleta" disminuyendo el acoplamiento y utilizando polimorfismo

- Favorecer la composición sobre herencia

Es fundamental para mejorar la flexibilidad y el mantenimiento del código. Aunque la herencia es una técnica útil para reutilizar código, en muchos casos puede generar problemas a largo plazo, especialmente cuando el sistema crece en complejidad. Este principio sugiere que, en lugar de depender de la herencia, se debe usar la **composición** para reutilizar comportamientos y características.



Herencia: (relación con open-close)

1. Fácil de modificar lo heredado:

Reutiliza código, pero los cambios en la clase padre afectan a todas las hijo.

2. Rompe el encapsulamiento:

Las subclases tienen acceso directo a los miembros internos de la clase padre.

3. Se define estáticamente:

La relación entre clases se determina en tiempo de compilación.

4. Caja blanca:

Los hijos conocen los detalles internos de la clase padre

Composición (agregación, asociación y composición) (porque gamma es viejo):

1. Comportamiento depende de las relaciones:

El comportamiento se basa en las relaciones con otros objetos.

2. Cambios en tiempos de ejecución:

El comportamiento puede modificarse dinámicamente al cambiar objetos relacionados.

3. No rompe el encapsulamiento:

Las clases solo interactúan a través de interfaces públicas, sin acceder a detalles internos.

4. Se define dinámicamente:

Las relaciones y comportamientos pueden cambiar en tiempo de ejecución.

5. Caja negra:

Los detalles internos de los objetos son ocultos; solo se interactúa a través de interfaces.

3. Principios SOLID

SOLID es un conjunto de principios que ayuda a organizar el código para que sea más fácil de mantener, flexible ante cambios y comprensible. Sin embargo, es importante aplicarlos con equilibrio y no forzar su implementación en todo momento. A veces, simplificar o ajustar según el contexto es la mejor opción. Además, **no** se conoce un producto de software exitoso que aplique todos estos principios al mismo tiempo

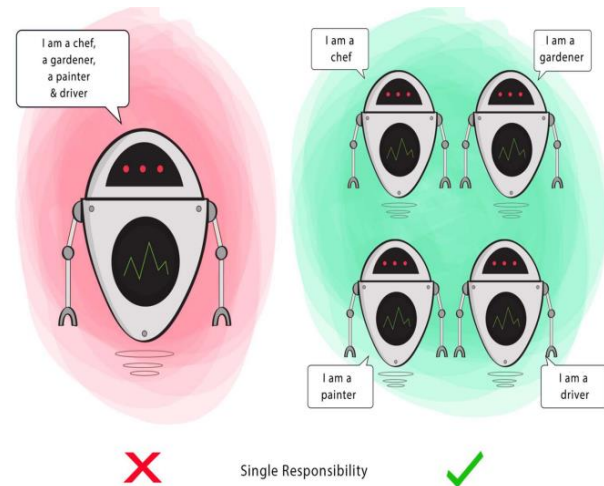


❖ Single Responsibility

Establece que cada clase debe tener una única responsabilidad, lo que significa que debe encargarse de una única tarea o conjunto de tareas estrechamente relacionadas. La idea es que, si una clase tiene demasiadas responsabilidades, se vuelve más difícil de mantener y modificar, ya que cualquier cambio en una de las responsabilidades podría afectar otras partes de la clase que no deberían verse afectadas.



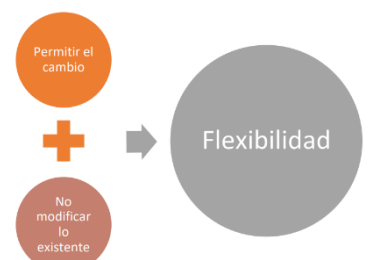
- **Reducir la complejidad:** Si una clase tiene varias responsabilidades, su código tiende a volverse más complejo y difícil de manejar. Al tener una única responsabilidad, es más fácil de entender y modificar.
- **Facilitar el mantenimiento:** Si una clase cambia debido a la modificación de una de sus responsabilidades, es probable que afecte otras responsabilidades dentro de la misma clase. Al seguir SRP, las modificaciones se pueden realizar sin comprometer otras funcionalidades.
- **Adapter, State, Iterator:** Estos patrones de diseño también promueven la separación de responsabilidades, asegurando que el código esté organizado de manera que sea fácil de extender sin modificar el código existente.
 - **Adapter:** Separa la responsabilidad de adaptar interfaces entre clases, permitiendo que cada clase mantenga su responsabilidad única sin modificar su código.
 - **State:** Divide el comportamiento en diferentes clases según el estado, delegando la responsabilidad de manejar cada estado a su propia clase, lo que mantiene el objeto original con una única responsabilidad.
 - **Iterator:** Separa la responsabilidad de recorrer una colección en un iterador, permitiendo que la colección se enfoque solo en almacenar los elementos.



❖ Open Close

Establece que las clases deben estar diseñadas de manera que **puedan extenderse** para agregar nuevas funcionalidades, pero **no deben modificarse** directamente. La idea principal es evitar que, al agregar nuevas funciones, se rompa o se afecte el comportamiento de código ya existente.

- **Abierta a la extensión:** Puedes crear subclases o nuevas clases que amplíen la funcionalidad de la clase base sin cambiar su código original. Esto permite agregar nuevas características sin alterar el comportamiento previamente establecido.
- **Cerrada a la modificación:** Una vez que la clase ha sido creada y probada, no deberías modificar su implementación interna. Si necesitas un comportamiento diferente o adicional, deberías hacerlo creando una nueva clase que extienda la clase base.



- **Adapter:** Abre la clase original a nuevas funcionalidades al crear una clase adaptadora sin modificar la clase base, manteniendo su comportamiento original intacto.
- **Observer:** Permite extender el sistema añadiendo nuevos observadores sin modificar las clases que ya observan, lo que hace que el sistema sea fácilmente extensible.
- **State:** Añadir nuevos estados a una clase es posible mediante la creación de nuevas clases de estado, sin alterar la clase original, lo que permite extender su comportamiento.
- **Strategy:** Se puede añadir nueva estrategia de comportamiento sin modificar las clases que usan la estrategia, solo creando nuevas clases que implementen la interfaz de estrategia.
- **Iterator:** Se extiende el comportamiento de iteración sin modificar la colección original, ya que el iterador puede ser modificado o reemplazado.

❖ Liskov Sustitution

Establece que **las clases “hijo” deben poder reemplazar a la clase “padre”** sin que el comportamiento esperado del sistema se vea alterado. Es decir, debes poder utilizar objetos “hijos” donde se espera un objeto de la clase padre, sin que se rompa el código que usa la clase “padre”.

- **State:** Las subclases de los estados de un objeto deben poder sustituir al estado base sin cambiar el comportamiento esperado del objeto.
- **Template Method:** El método base proporciona una plantilla que puede ser extendida por las subclases sin modificar su estructura fundamental. Las subclases añaden o modifican pasos específicos, pero no alteran el flujo general del proceso.



❖ Interface Segregation (“no se debe forzar a los clientes a depender de métodos que no utilizan”)

Problema:

Interfaces demasiado grandes o generales obligan a las clases concretas a implementar métodos que no necesitan. Esto genera bajo nivel de cohesión y dificultad para mantener el código.

Solución:

Dividir las interfaces grandes en interfaces más pequeñas y específicas. Cada interfaz debe ser altamente cohesiva y ajustarse a las necesidades de sus clientes. Aunque se generen más interfaces, serán más manejables y claras.

Conceptos clave:

- **Interfaz:** Define contratos (nombre del método, parámetros y retorno) sin implementación; especifica qué debe hacerse, pero no cómo.
- **Clase Abstracta:** Puede contener métodos abstractos (sin implementación) y métodos concretos. NO se pueden instanciar
- **Método abstracto:** Declarado, pero no implementado.
- **Método concreto:** Contiene implementación.
- **Relación de Realización:** Conexión entre clases concretas y las interfaces que implementan.

Ventajas:

1. Las clases concretas solo implementan métodos que realmente necesitan.
2. Aumenta la cohesión y facilita el mantenimiento del código.
3. Reduce el acoplamiento entre clases y métodos innecesarios

Patrones relacionados:

- **Adapter:** Usa interfaces específicas para conectar clases incompatibles.
- **Observer:** Notifica solo a los interesados, sin métodos adicionales.
- **Strategy:** Permite intercambiar algoritmos mediante interfaces específicas.
- **Iterator:** Define una interfaz para recorrer colecciones sin exponer detalles internos.

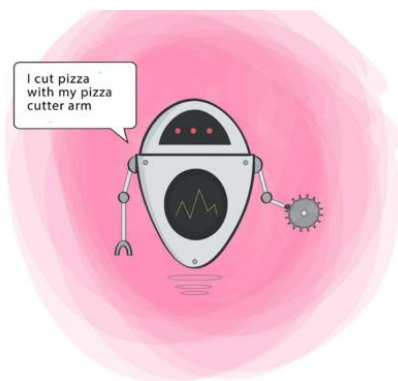
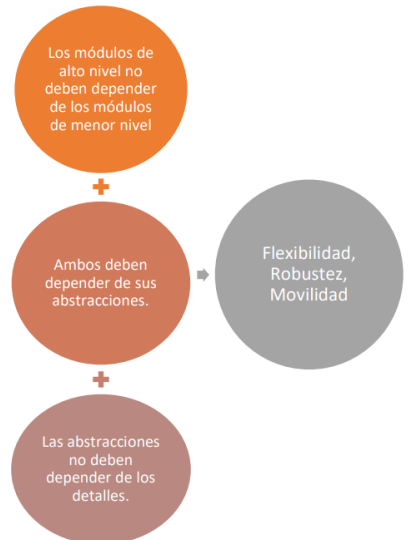
❖ Dependency Inversión (Inversión de Dependencias)

(Hollywood, “no nos llames, nosotros te llamamos”)

Busca desacoplar el código y reducir las dependencias entre las clases, especialmente entre las de **alto nivel** (que contienen la lógica de negocio) y las de **bajo nivel** (que realizan operaciones más básicas, como acceder a bases de datos, gestionar redes, etc.).

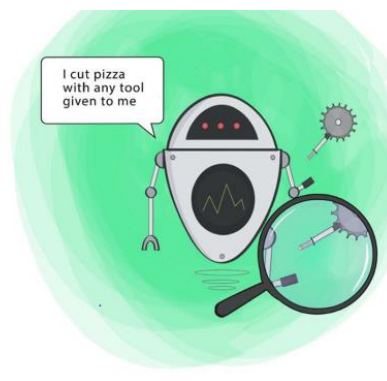
Para aplicar este principio, generalmente seguimos estos pasos:

1. **Definir interfaces** para las operaciones de bajo nivel que usará la lógica de negocio. Estas interfaces deben ser representadas en términos de lo que el negocio necesita hacer, no en detalles técnicos específicos (como acceder a una base de datos o escribir en un archivo).
2. **Las clases de alto nivel** (la lógica de negocio) deben depender de estas interfaces, no de clases concretas de bajo nivel. Esto hace que las clases de alto nivel no estén directamente acopladas a las implementaciones de bajo nivel.
3. **Las clases de bajo nivel** deben implementar estas interfaces. De esta forma, las clases de bajo nivel se vuelven dependientes de las abstracciones, y no al revés, como ocurriría en un diseño tradicional.



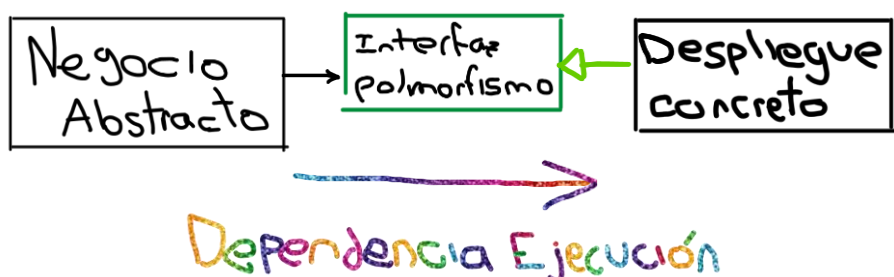
✗

Dependency Inversion



✓

En este ejemplo, se hace referencia a que una clase de alto nivel (el robot) no debería acoplarse a una clase de bajo nivel (la cuchilla). Sino que se debería usar una abstracción (interfaz) que va a permitir utilizar la herramienta que necesite dependiendo la situación. Aumentando la flexibilidad, disminuyendo la robustez y aumentando la movilidad.

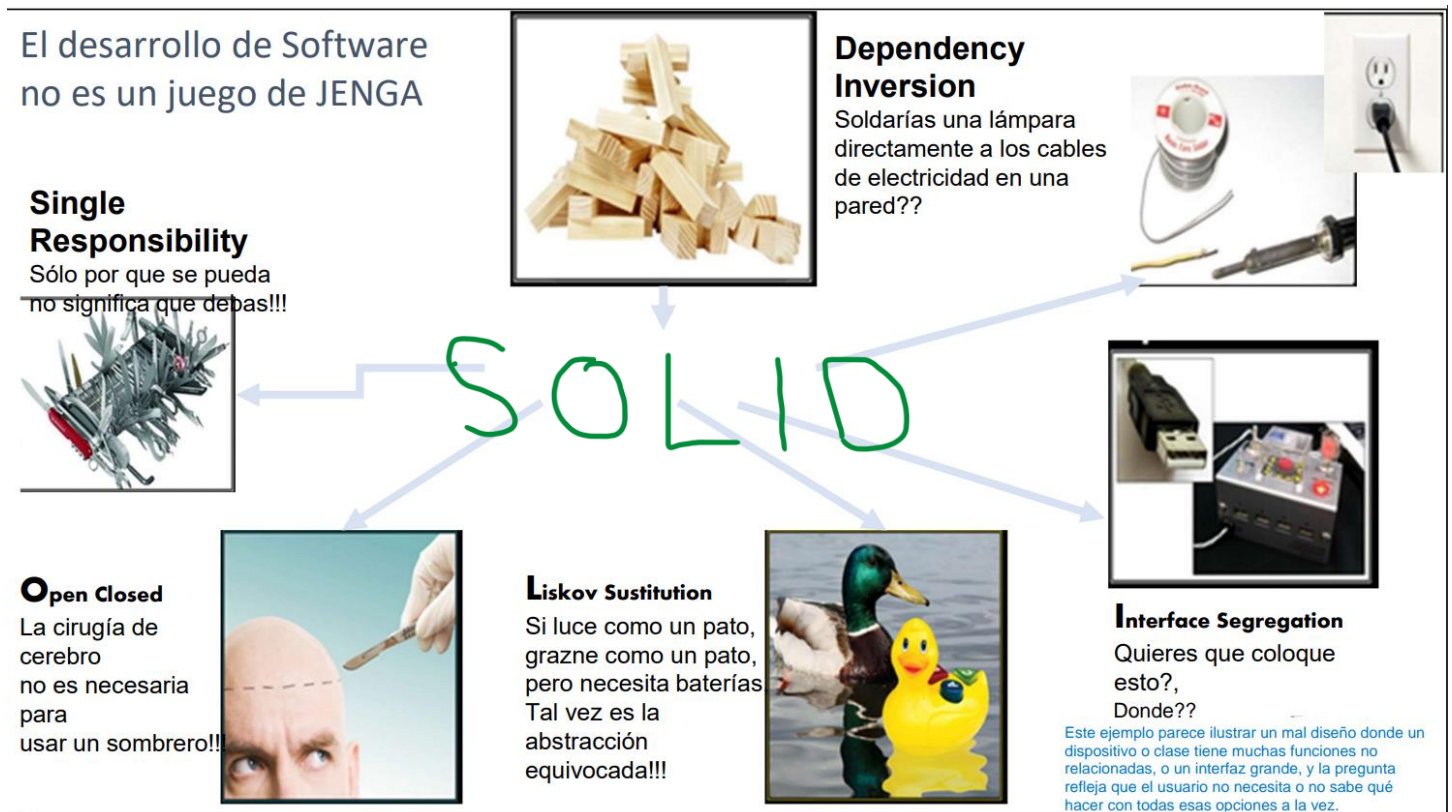


Patrón Relacionado:

- **Template Method** es un patrón de diseño que se relaciona estrechamente con el principio de **Inversión de Dependencias** (DIP) porque permite que una clase de alto nivel defina la estructura general de un algoritmo, mientras que las clases de bajo nivel proporcionan las

implementaciones específicas de ciertos pasos del algoritmo a través de interfaces o métodos abstractos. Esto cumple con el principio de que las clases de alto nivel no deben depender de detalles concretos, sino de abstracciones (en este caso, interfaces o clases abstractas)

Resumen:



4. Conceptos Importantes

- **Cohesión:** Medida de fuerza de relación interna. El precio que pagamos para una mayor cohesión es mayor cantidad de componentes.
- **Acoplamiento:** Medida de relación externa. Como una clase se relaciona con las demás
 - Tanto la **cohesión** como el **acoplamiento** se aplican a clases, capas, subsistemas, servidores, interfaces.



- **Herencia** → Herencia de implementación
- **Realización** → Herencia de interfaces
- **Asociación** → Relación del todo con sus partes, no tan estricta como la composición
- **Composición** → Relación más estricta que hay
- **Delegación.**

Gamma se refiere como COMPOSICION a estos 3

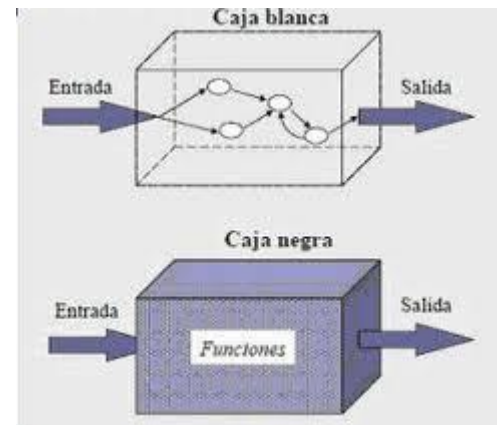
Es el proceso de transferir la responsabilidad de la ejecución de una tarea a otro objeto. En vez de que una clase realice una operación directamente, delega esa operación a una clase diferente que es más adecuada para manejarla. Es una técnica que promueve la **separación de responsabilidades** y la **reutilización del código**.

- **Polimorfismo.**

Es un concepto fundamental en la programación orientada a objetos que permite que un objeto se comporte de diferentes maneras, dependiendo de su tipo o la clase que lo implemente. Es cuando una o varias clases hija sobrescriben un método de la clase padre para implementar un comportamiento diferente, pero sigue usando la misma interfaz o nombre del método.

Reúso de Caja Blanca

- **Acceso a la estructura interna:** En el reúso de caja blanca, los desarrolladores pueden ver y modificar el código fuente del componente reutilizado. Este tipo de reúso permite un alto grado de personalización y ajuste de la lógica interna.
- **Ejemplo en herencia:** Este enfoque es común en la herencia de clases, donde se accede al comportamiento y estructura interna de una clase base, permitiendo extender o sobrescribir métodos en las clases derivadas. Es típico en Programación Orientada a Objetos (POO), ya que el desarrollador modifica o especializa el comportamiento heredado.
- **Ventajas:** Ofrece flexibilidad y control sobre la funcionalidad del componente, permitiendo modificaciones profundas y adaptaciones específicas a las necesidades del proyecto.
- **Desventajas:** Genera un alto acoplamiento, ya que cualquier cambio en el componente reutilizado afecta directamente al sistema que depende de él. También puede requerir mayor conocimiento del código y aumentar la complejidad.



Reúso de Caja Negra

- **Acceso limitado a la estructura interna:** En el reúso de caja negra, los desarrolladores no necesitan (ni pueden) ver ni modificar el código interno del componente. Solo se conoce y utiliza la interfaz pública para interactuar con él, sin acceso a su implementación.
- **Ejemplo en composición o servicios:** Un ejemplo es el uso de bibliotecas o servicios externos donde se interactúa mediante una interfaz o API bien definida, sin necesidad de saber cómo está implementada internamente. La composición y el uso de interfaces en POO también aplican este enfoque.
- **Ventajas:** Reduce el acoplamiento, ya que el sistema depende solo de la interfaz del componente y no de su implementación. Esto mejora la modularidad y facilita la sustitución de componentes sin afectar al sistema.
- **Desventajas:** Ofrece menos flexibilidad, ya que el componente reutilizado no se puede modificar internamente. Esto puede limitar la capacidad de ajuste si el componente no cubre exactamente todas las necesidades.

5. Evolución de software

La evolución del software es esencial porque los sistemas deben adaptarse constantemente para seguir siendo útiles y eficaces en entornos cambiantes. Esto implica la necesidad de:

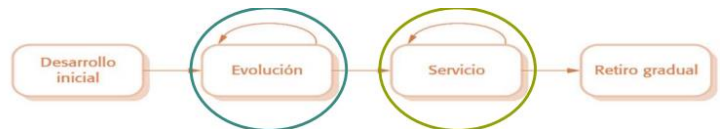
1. **Adaptación al entorno:** Los cambios en hardware, software o normativas externas exigen ajustes en los sistemas.
2. **Evitar obsolescencia:** Los sistemas que no se actualizan se vuelven difíciles de mantener y menos relevantes para las necesidades actuales.
3. **Complejidad creciente:** Con el tiempo, realizar cambios en un sistema puede volverse más difícil y costoso debido a la degradación de su estructura.

El mantenimiento y evolución del software deben considerarse como un **proceso iterativo y continuo**, siguiendo un modelo como el **espiral**, en el cual las fases de requerimientos, implementación, validación y operación se repiten a lo largo del ciclo de vida del sistema.

Puede haber **una sola organización** responsable del desarrollo inicial como de la evolución de software. Si las compañías responsables son distintas, habrá discontinuidad en el proceso de espiral.



Distinguir entre Evolución y Servicio

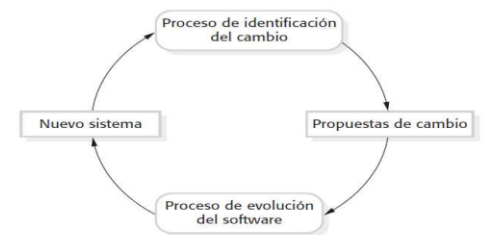


Evolución: Fase donde es posible realizar cambios significativos a la arquitectura y funcionalidad del software

Servicio: En esta fase los únicos cambios que se realizan son relativamente pequeños

Procesos de evolución

La evolución de un sistema de software se impulsa principalmente por **propuestas de cambio**, que son el motor de este proceso y responden a diversas necesidades. Estas propuestas pueden surgir de **nuevos requerimientos no anticipados, reportes de errores o fallas, ideas para mejorar el sistema**.



Los **procesos de identificación de cambios y evolución del sistema** son cíclicos a lo largo de vida de un sistema.



Como se aprecia en el diagrama, al recibir una petición de cambio, lo primero que se hace es evaluar el impacto que tendrá el mismo en el sistema, luego se planifican las versiones, lo cual implica reparar los defectos, adaptar la plataforma y perfeccionar el sistema, de ahí se pasa a implementar estos cambios en el sistema, donde podría surgir un nuevo inconveniente el cual se debe resolver, por último se realiza la entrega del sistema, estando siempre listo para recibir peticiones de cambio.

Etapas de implementación de cambios → puede considerarse una iteración del proceso de desarrollo, donde las revisiones se diseñan, se aplican y se ponen a prueba. La diferencia es que puede involucrar la comprensión del programa

Reparación de Emergencia

- Falla seria del sistema
 - Cambios a los sistemas que operan el entorno tienen efectos inesperados
 - Cambios no anticipados (nuevos competidores, nueva legislación)
-

Uso de métodos ágiles en la evolución

- **Continuidad del desarrollo ágil:** Cuando un software necesita evolucionar (por ejemplo, adaptarse a nuevos requisitos, corregir errores o mejorar funcionalidades), idealmente el equipo que se encarga de la evolución debería continuar con los mismos principios y métodos ágiles usados durante el desarrollo.
- En los métodos ágiles, se prioriza la **colaboración continua**, la **adaptación al cambio** y la **entrega incremental**. Esto permite que las actualizaciones sean más rápidas y flexibles.

Problemas en la transferencia de responsabilidades

Existen problemas que pueden surgir cuando el equipo que desarrolló el software transfiere la responsabilidad de su evolución a otro equipo (común en empresas grandes o proyectos complejos). Estas tensiones suelen ocurrir porque ambos equipos pueden trabajar con **enfoques diferentes**.

a. Conflicto entre enfoques

- Si el **equipo de desarrollo** trabaja con un enfoque ágil, pero el **equipo de evolución** utiliza un enfoque más **basado en planes (plan-driven)**, puede haber desacuerdos.
 - El **enfoque ágil** busca adaptarse rápidamente a los cambios, mientras que el **enfoque basado en planes** prioriza la planificación detallada antes de ejecutar tareas.
 - Ejemplo de problema: El equipo de evolución podría pedir documentación exhaustiva, mientras que el equipo ágil priorizó la entrega rápida y no produjo tanta documentación.
- Lo contrario también puede ser problemático: si el **equipo de desarrollo** trabajó con un enfoque basado en planes, pero el **equipo de evolución** prefiere métodos ágiles.
 - Aquí, el equipo de evolución podría sentirse limitado por procesos rígidos y una planificación demasiado detallada.
- Las reparaciones rápidas suelen priorizar la velocidad sobre la calidad, lo que puede generar problemas futuros y una degradación estructural del sistema.

b. Desafíos organizacionales

- Estos conflictos suelen reflejarse en desacuerdos sobre la forma de trabajo, tiempos de entrega y prioridades.
- Sin un entendimiento mutuo o un esfuerzo por alinear las metodologías, el trabajo puede volverse ineficiente y generar retrasos.

En cualquier caso, los cambios realizados deben documentarse y revisarse posteriormente para garantizar que el sistema mantenga su consistencia y facilidad de mantenimiento.

Dinámica de la evolución del software

Propuesta por **Lehman**, describe cómo los sistemas de software evolucionan a lo largo del tiempo. Aplicables para todos los tipos de sistemas de software organizacional grandes, en los cuales los requerimientos se modifican para reflejar las necesidades cambiantes de la organización. Las nuevas versiones son esenciales para que el sistema proporcione valor al negocio

Leyes de Lehman:

1. Cambio continuo:

- Los sistemas que se utilizan en entornos reales deben evolucionar constantemente para adaptarse a nuevas necesidades o entornos. Si no se actualizan, se vuelven obsoletos y menos útiles para los usuarios.

2. Complejidad creciente:

- A medida que el software evoluciona, su estructura tiende a volverse más compleja. Esto requiere esfuerzos adicionales para mantenerlo comprensible y evitar que se convierta en un sistema inmanejable.

3. Evolución de programas grandes:

- Los programas evolucionan siguiendo un proceso autorregulado. Esto significa que, independientemente de los recursos dedicados, factores como el tamaño, el tiempo de entrega y la cantidad de errores se mantienen dentro de ciertos rangos predecibles.

4. Estabilidad organizacional:

- La velocidad de desarrollo de un sistema tiende a permanecer estable en el tiempo, sin importar cuántos recursos se asignen. Esto implica que hay un límite natural en la capacidad de evolución de un sistema.

5. Conservación de la familiaridad:

- Los desarrolladores y usuarios deben conservar el conocimiento del sistema a medida que este evoluciona. Si el sistema crece demasiado rápido o de forma desordenada, se pierde esa familiaridad, dificultando su mantenimiento.

6. Crecimiento continuado:

- Para satisfacer las demandas de los usuarios, un sistema debe expandir continuamente su funcionalidad. Esto implica que no solo se deben resolver errores, sino también agregar nuevas características.

7. Declive de la calidad:

- Si un sistema no se adapta adecuadamente a los cambios en su entorno, su calidad inevitablemente disminuirá. Esto incluye problemas de rendimiento, errores o falta de compatibilidad con nuevas tecnologías.

8. Sistema de realimentación:

- Los procesos de evolución deben incorporar retroalimentación continua (como la interacción con usuarios y desarrolladores) para mejorar el sistema. Esta retroalimentación asegura que el producto siga siendo relevante y útil.

Estas leyes destacan que la evolución del software no es solo un proceso técnico, sino también organizacional y social. Ignorar estas dinámicas puede llevar a que el sistema se vuelva ineficiente, difícil de mantener o irrelevante. Por lo tanto, es crucial planificar la evolución del software considerando estos principios para garantizar su sostenibilidad a largo plazo.

Mantenimiento del software

Es el proceso de realizar cambios en un sistema después de su entrega.

Tipos de mantenimiento:

1. **Correctivo:** Reparación de fallas en diseño, código (los más baratos) o requerimientos.
2. **Adaptativo:** Cambios en el sistema para adaptarlo a un entorno diferente.
3. **Perfectivo:** Mejora o adición de funcionalidades.

Desafíos en el mantenimiento:

- **Mantenimiento y costo:** Desarrollar software pensando en su mantenibilidad reduce costos a largo plazo. Esto se refiere a crear sistemas que puedan actualizarse o mejorarse con facilidad, lo cual es fundamental para que el software evolucione con las necesidades de los usuarios.
- **Refactorización en desarrollo ágil:** En metodologías ágiles, la refactorización (mejorar el código sin cambiar su funcionalidad) es una práctica habitual. Esto permite que el software evolucione de manera continua mientras se adapta a cambios rápidos.
- **Desarrollo basado en plan:** Es común que no se prioricen mejoras al código durante la fase de desarrollo. Esto puede hacer que la evolución del software sea más costosa o difícil en el futuro.



Predicción de mantenimiento

Es crucial para anticiparse a los cambios necesarios y minimizar costos. Factores a considerar:

1. **Predicción de cambios al sistema:** Se enfoca en anticipar qué cambios podrían proponerse al sistema y cuántas solicitudes de modificación pueden esperarse. Esto ayuda a planificar la capacidad de adaptación del software frente a cambios futuros.
2. **Predicción de mantenibilidad:** Identifica las partes del sistema que podrían ser más difíciles de mantener. Esto incluye evaluar el diseño del sistema y su facilidad para incorporar actualizaciones sin comprometer la funcionalidad.
3. **Predicción de costos de mantenimiento:** Analiza los costos proyectados durante la vida útil del sistema, considerando tanto el mantenimiento a corto plazo como en el próximo año. Esto permite prever los recursos necesarios para su sostenibilidad.

Para realizar estas predicciones, es crucial entender cómo el sistema interactúa con su entorno externo. Esto incluye:

- Evaluar el número y la complejidad de las interfaces del sistema con otros componentes o sistemas.
- Identificar los requerimientos que son inestables por naturaleza y que pueden cambiar frecuentemente.
- Analizar los procesos empresariales en los que el sistema se utiliza, ya que estos pueden impactar directamente en su necesidad de mantenimiento.

Reingeniería del software

Es el proceso de rediseñar un sistema existente para mejorar su estructura y entendimiento, adaptándolo a nuevas necesidades. Suele implicar:

- **Documentar nuevamente el sistema**
- **Refactorizar su arquitectura**
- **Traducción a un nuevo lenguaje**
- **Modificar y actualizar la estructura y valores de datos del sistema**

NO CAMBIA LA FUNCIONALIDAD DEL SISTEMA.

La **reingeniería de software** es el proceso de analizar, modificar y reconstruir un sistema existente para mejorar su calidad, mantenibilidad o adaptarlo a nuevos requisitos. La imagen muestra su proceso, junto con los beneficios frente a sustituir el software.



Proceso de reingeniería:

1. **Traducción del código fuente:** Convertir el código a un nuevo lenguaje
2. **Ingeniería inversa:** Analizar el programa para entenderlo
3. **Mejora de la estructura de los programas:** Reestructurar de forma automática para la comprensibilidad
4. **Modularización de los programas:** Reorganizar la estructura de los programas
5. **Re-ingeniería de datos:** Limpiar y re-estructurar los datos del sistema

Beneficios de la reingeniería frente a la sustitución:

- **Reducción del riesgo:**
 - Eliminar y crear un nuevo sistema desde cero implica altos riesgos, como errores en la especificación o problemas de desarrollo. La reingeniería minimiza estos riesgos al trabajar sobre una base existente.
- **Reducción de costos:**
 - Reestructurar y mejorar un programa suele ser mucho más económico que desarrollar un software completamente nuevo.

Refactorización (mantenimiento preventivo)

La refactorización mejora la estructura del sistema sin modificar su funcionalidad, lo que reduce los costos de mantenimiento a largo plazo. Sirve para prevenir la degradación del software causada por cambios continuos.

Aspectos que se pueden mejorar mediante refactorización:

1. **Código duplicado.** (principio DRY)
2. **Métodos largos.** (principio KISS)
3. **Uso excesivo de switch (case)**
4. **Aglomeración de datos.**
5. **Generalidad especulativa** (elementos innecesarios "por si acaso") (principio YAGNI)

Reingeniería	Refactorización
Se realiza después de haber mantenido un sistema un tiempo.	Proceso continuo de mejoramiento debido al proceso de desarrollo y evolución.
Dicho mantenimiento implica aumento de los costos.	Procura evitar la degradación de la estructura y el código que aumentan los costos de mantenimiento.
Uso de herramientas automatizadas para procesar y someter a reingeniería a un sistema heredado.	Parte inherente de los métodos ágiles.
Creación de un nuevo sistema más mantenible.	No implica crear un nuevo sistema.

Administración de sistemas heredados

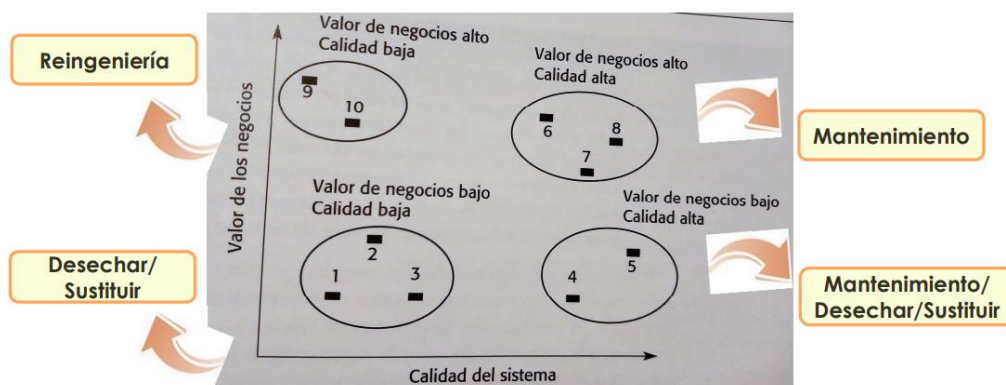
Los sistemas heredados son software antiguo que sigue siendo crítico para la organización. Las estrategias de administración incluyen:

1. **Desechar el sistema:** Cuando es obsoleto y no aporta valor.
2. **Dejarlo sin cambios:** Si es estable y sigue cumpliendo con su propósito.
3. **Reingeniería:** Para mejorar su calidad y funcionalidad.
4. **Sustitución parcial o total:** Si ya no es viable seguir usándolo.

Valoración de un sistema heredado:

1. **Valor empresarial:** Determina si el sistema sigue siendo valioso para la organización.
2. **Valor desde la perspectiva técnica:** Evalúa la calidad técnica de la aplicación en sí, considerando factores como hardware y software
3. **Valorar la calidad técnica (confiabilidad, facilidad de mantenimiento, documentación):** Nro. de peticiones de cambio, nro. de interfaces de usuario, volumen de datos usado por el sistema

Clasificación de sistemas:



6. Persistencia

La **persistencia** es la capacidad de que los objetos creados en una aplicación sobrevivan más allá de la ejecución del programa que los generó. Esto implica mover los datos desde la memoria primaria (**RAM**, volátil y rápida) a la secundaria (**disco**, persistente y más lenta).

El diseño de persistencia no solo consiste en decidir dónde guardar los datos, sino también en cómo estructurarlos para garantizar su coherencia y eficiencia. Este diseño debe abordar dos aspectos fundamentales:

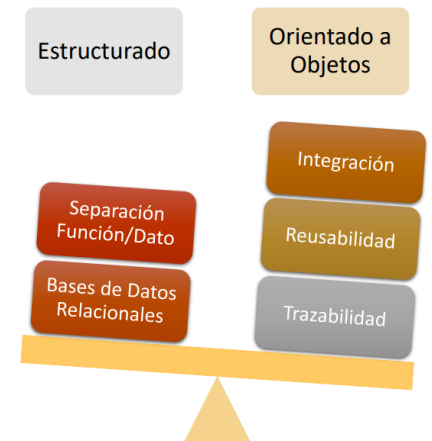
1. **Análisis y selección de clases persistentes:** Determinar qué objetos o entidades del sistema necesitan ser almacenados.
2. **Diseño de la base de datos:** Decidir cómo las clases persistentes se traducirán en estructuras relacionales como tablas, columnas y relaciones.

El diseño de persistencia se conceptualiza en la arquitectura del sistema como un **componente de persistencia**. Este componente actúa como un puente entre la **lógica de negocio**, que encapsula las reglas y procesos del sistema, y la **base de datos relacional**, que almacena la información en estructuras definidas por el modelo relacional.

Paradigmas

1. Estructurado:

- **Separación Función/Dato:** En los sistemas estructurados, los datos y las funciones que los manipulan están separados. Los datos son almacenados en **tablas** (como en bases de datos relacionales), y las funciones se aplican sobre esos datos. Esto permite un control más claro y organizado de la información.
- **Base de Datos Relacionales:** Utilizan tablas para almacenar datos, donde cada tabla tiene filas y columnas, y las relaciones entre ellas se gestionan mediante claves primarias y foráneas. Se aplica el **modelo relacional** para organizar y consultar los datos.



2. Orientado a Objetos:

- **Integración:** Los sistemas orientados a objetos permiten integrar funciones y datos en un único objeto. Cada objeto tiene **atributos** (datos) y **métodos** (funciones), lo que facilita la interacción entre distintas partes del sistema.
- **Reusabilidad:** Gracias a la **herencia** y la **composición**, los objetos y clases pueden ser reutilizados en diferentes partes del sistema o en proyectos futuros, lo que ahorra tiempo y esfuerzo en el desarrollo.
- **Trazabilidad:** Este paradigma facilita la **trazabilidad** de las relaciones entre objetos, clases y procesos. Como los objetos están bien definidos, se puede rastrear cómo los datos se transforman y se utilizan a lo largo del sistema.

Reglas de integridad

Al diseñar persistencia en bases de datos relacionales, es crucial respetar las reglas de integridad:

- **Integridad relacional:**
 - **Unicidad:** Cada tupla debe tener una clave única (PK), asegurando que no haya duplicados.
 - **Minimalidad:** Las claves deben ser lo más simples posible, sin incluir atributos redundantes.
- **Integridad referencial:**
 - Garantiza que los valores de claves foráneas (FK) coincidan con claves primarias existentes en las tablas relacionadas.

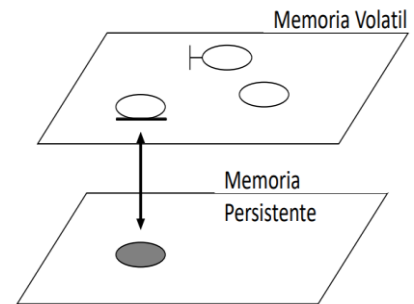
Problemática del diseño de persistencia

El principal desafío surge de la diferencia entre los paradigmas utilizados en programación y bases de datos:

- **Programación Orientada a Objetos (POO):** Los objetos combinan datos (atributos) y comportamiento (métodos). Además, cada objeto tiene identidad propia, basada en su dirección en memoria.
- **Modelo Relacional:** Se enfoca en almacenar y relacionar datos en estructuras estáticas (tablas), sin incluir comportamiento. La identidad de una tupla está definida por su **clave primaria**.

Esto genera un **problema de impedancia**:

1. Los comportamientos de los objetos no pueden almacenarse directamente en una base de datos relacional.
2. La identidad de los objetos en memoria (por referencia) no coincide con la identidad de las tuplas en una base de datos relacional (por clave primaria).
3. Tipos de datos complejos de los objetos deben ser transformados a tipos primitivos compatibles con el DBMS.



Los círculos en la capa de memoria volátil simbolizan objetos que existen temporalmente mientras el programa está en ejecución, pero que desaparecen cuando este finaliza. El círculo oscuro conectado a la memoria persistente indica que ese dato u objeto ha sido guardado en una capa de almacenamiento permanente, asegurando que su estado se conserve incluso después de apagar el sistema o cerrar el programa.

Mapeo de clases a bases de datos relacionales

El **Object-Relational Mapping (ORM)** es la técnica usada para superar la impedancia entre POO y bases de datos relacionales. Este proceso incluye:

1. **Selección de clases persistentes:** Se descartan clases que no necesitan almacenamiento (clases de utilidades o de control).
 - Las relaciones que necesitan persistir son: **asociación** (agregación y composición) y **generalización** (herencia)
2. **Conversión a tablas:**
 - Cada clase persistente del modelo de dominio se convierte en una tabla.
 - Los atributos de la clase se mapean a columnas de la tabla.
 - Se asigna una clave primaria para identificar de forma única cada fila.
3. **Eliminación del comportamiento:** Dado que las bases de datos relacionales no almacenan métodos, solo se persisten los atributos.
4. **Trazabilidad:** La tabla conserva el nombre de la clase para facilitar el mantenimiento y la comprensión del diseño.

Relaciones entre clases que requieren persistencia

El diseño de persistencia para relaciones entre clases requiere mapear las relaciones de **asociación**, (agregación y composición incluidas) y **herencia** a estructuras relacionales que respeten las reglas de integridad del modelo de datos.

Tipos de relaciones entre clases:

1. Asociación

Es una relación estructural básica entre dos entidades. Representa cómo los objetos de una clase se relacionan con los de otra clase.

2. Agregación

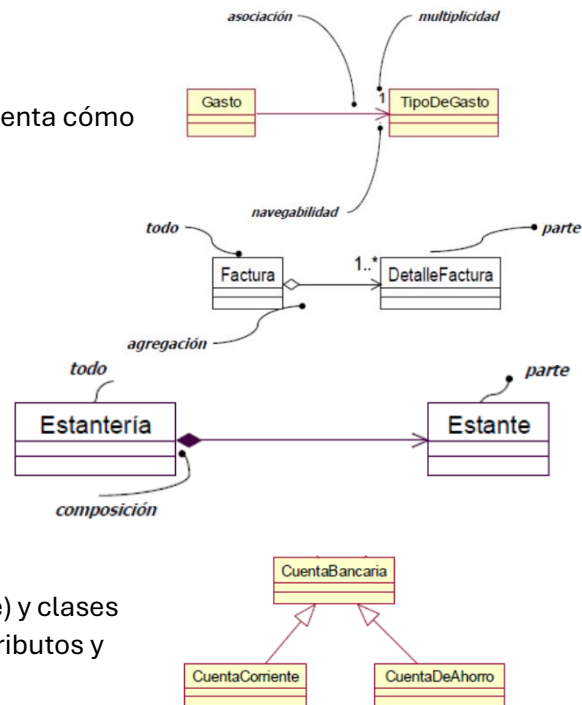
Es una relación conceptual de "todo-parte". Aunque las partes pueden existir independientemente del todo, hay una relación significativa entre ellas.

3. Composición

Es una variante más fuerte de la agregación, donde las partes **no pueden existir** sin el todo, y sus ciclos de vida están vinculados.

4. Generalización (Herencia)

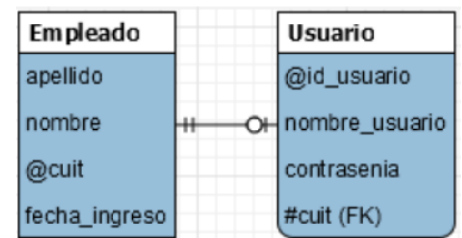
Representa una relación "es un" entre una clase general (padre) y clases más específicas (hijas). Las hijas pueden añadir o modificar atributos y métodos del padre.



Mapeo de asociaciones en bases de datos

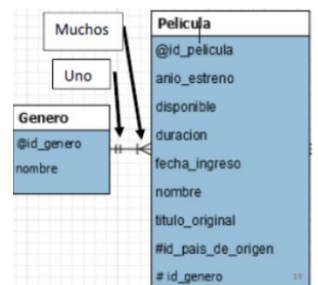
1. Uno a uno (1:1)

- Cada instancia de una tabla se asocia con exactamente una instancia de otra tabla.
- Implementación:
 - Usamos claves primarias y foráneas para conectar ambas tablas.
 - Si ambas entidades tienen opcionalidad 1 (relación 0...1 en ambos lados), podemos combinar ambas tablas en una sola para simplificar.



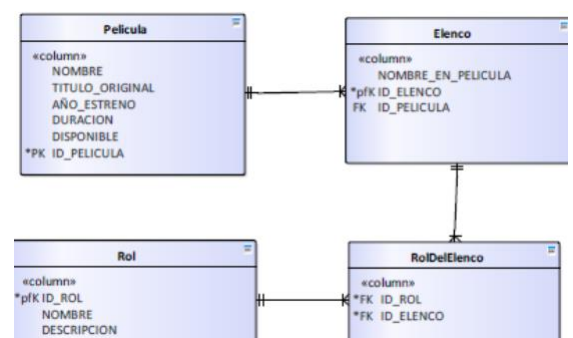
2. Uno a muchos (1: N) o Muchos a uno (N:1)

- Ejemplo:
 - Uno a muchos:** Un género puede estar asociado con varias películas.
 - Muchos a uno:** Varias películas tienen un solo género.
- Implementación:
 - Se utiliza una clave foránea en la tabla del lado "muchos" que apunte a la clave primaria del lado "uno".



3. Muchos a muchos (M:N)

- Ocurre cuando múltiples instancias de una tabla están relacionadas con múltiples instancias de otra tabla.
- Problema: Esto puede generar violaciones de integridad al repetirse claves primarias.
 - Solución: Introducir una tabla intermedia para representar la relación.



RolDelElenco sería la tabla intermedia

Mapeo de herencia

Las relaciones de herencia en POO no tienen un equivalente directo en bases de datos relacionales. Existen tres enfoques comunes para manejarlas:

1. Jerarquías supertipo-subtipo

- Se crean tablas separadas para la clase padre y las clases hijas.
- La tabla padre contiene los atributos comunes.
- Las tablas hijas tienen una relación uno a uno con la tabla padre y contienen atributos específicos.

2. Eliminación del padre

- Los atributos del padre se distribuyen entre las tablas hijas.
- Se eliminan las relaciones de herencia, pero aumenta el tamaño de las tablas debido a la duplicación de atributos comunes.

3. Eliminación de los hijos

- Se combinan todos los atributos de las clases hijas con los del padre en una sola tabla.
- Implementación: Se agrega una columna adicional para identificar el tipo de cada fila (discriminador).
- Desventaja: **Poca cohesión** y posible **desperdicio** de espacio por atributos no utilizados.

Modelos de Persistencia

Los modelos de persistencia son fundamentales en el diseño de software para gestionar cómo los objetos en memoria (clases del sistema) se almacenan y recuperan desde una base de datos. Estos modelos encapsulan la lógica necesaria para que los objetos puedan persistir más allá de la ejecución de una aplicación.

Se creará un **framework de persistencia**, una herramienta o biblioteca que simplifica el proceso de almacenar y recuperar objetos desde una base de datos.

Las funciones que deben proporcionar se conocen como materialización y desmaterialización. La materialización se refiere a convertir uno o varios registros de la base de datos en objetos en memoria, mientras que la desmaterialización se refiere a grabar un objeto en la base de datos en forma de uno o varios registros de tablas.

Super Objeto Persistente

Todas las clases que necesitan persistencia **heredan** de una clase base llamada "súper objeto persistente". Este súper objeto contiene el comportamiento necesario para materializar y desmaterializar los objetos, permitiendo su almacenamiento en la base de datos. Se suele usar en proyectos pequeños o de corta vida útil donde la simplicidad prima sobre la mantenibilidad y extensibilidad.

Ventajas:

- **Fácil de implementar:** Agregar una clase base simplifica la estructura y el manejo de persistencia.
- **Estructura compacta:** No aumenta significativamente el tamaño del modelo.

Desventajas:

- **Rompe principios SOLID:**
 - **SRP:** Las entidades tienen múltiples responsabilidades (persistencia y negocio).

- **Open-Close:** Las clases deben modificarse para heredar del súper objeto, lo que implica violar el principio.
- **Reducción de cohesión:** Las clases de dominio asumen responsabilidades que no son propias.
- **Alto acoplamiento:** Dependencia directa entre la lógica de negocio y la capa de persistencia.
- **Problemas de pruebas:** Dificulta las pruebas unitarias debido al acoplamiento con la base de datos.

Esquema de Persistencia

El **esquema de persistencia** es una solución que utiliza la **correspondencia indirecta** (enfoque para interactuar entre dos sistemas que no son compatibles de forma directa) para resolver el problema del alto acoplamiento entre las capas de lógica de negocio y de datos. Este esquema introduce una **capa intermedia de persistencia**, compuesta por **clases de fabricación pura**, que actúan como puente entre las clases del dominio y la base de datos.

Funcionamiento

1. Clases de fabricación pura:

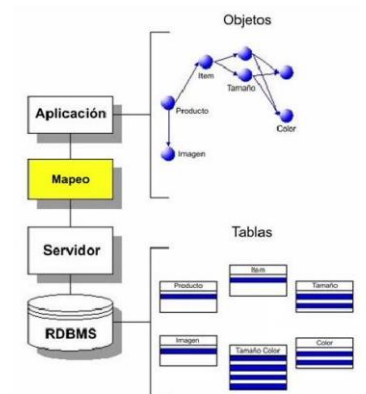
- Se crea una para cada clase de entidad que necesita persistencia.
- Estas clases manejan las operaciones de **materialización** (convertir datos de la base en objetos) y **desmaterialización** (guardar objetos en la base).

2. Independencia de las clases de dominio:

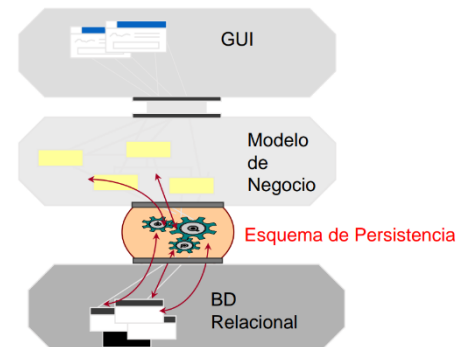
- Las clases del dominio no interactúan directamente con la base de datos.
- Esto asegura que las entidades no se vean afectadas por los detalles de persistencia.

3. Uso del principio de inversión de dependencia:

- Los objetos de la lógica de negocio (alto nivel) no dependen de los niveles inferiores (como la base de datos) (bajo nivel).
- El esquema sigue el principio "Hollywood": *"No nos llames, nosotros te llamamos"*, donde la capa de persistencia maneja las solicitudes.



El Esquema de persistencia sería el mapeo



Ventajas

- **Mantiene la cohesión:** Las clases del dominio se centran en su lógica y no incorporan detalles de persistencia.
- **Extensibilidad y reusabilidad:** Facilita la evolución y reutilización del sistema. Es más reusable porque separa la lógica de negocio de los detalles de persistencia, permitiendo que las clases de negocio se mantengan independientes de la base de datos. Al centralizar la lógica de persistencia en una capa intermedia, se facilita el cambio de mecanismos de almacenamiento sin afectar las clases de negocio. Además, al seguir principios como la inversión de dependencias, se garantiza que la persistencia sea fácilmente extensible y adaptable a nuevos requerimientos sin modificar el código existente.
- **Cumple con principios SOLID:**

- **Responsabilidad única:** La lógica de negocio y la persistencia están separadas.
- **Open/Close:** Las clases de dominio no se modifican para añadir funcionalidad.

Desventajas

- **Incremento en el tamaño del modelo:**
 - La necesidad de crear una clase de fabricación pura por cada clase persistente duplica la cantidad de clases, lo que aumenta la complejidad del sistema.
 - **Crecimiento del sistema:**
 - La capa de persistencia puede hacer que la estructura general sea más difícil de gestionar en sistemas grandes.
-

Características de las BDOO

Las **BDOO** se distinguen de las bases de datos relacionales al soportar las características de la programación orientada a objetos y proporcionar capacidades adicionales que permiten manejar objetos complejos y relaciones más naturales entre datos. Estas son algunas de las características clave que definen a las BDOO:

1. Objetos Complejos:

- Capaces de manejar objetos complejos, que pueden contener otros objetos como atributos (en lugar de solo datos atómicos como en las bases de datos relacionales).

2. Identidad de Objetos:

- Cada objeto en la base de datos debe tener una identidad única, que se mantiene independientemente de sus valores internos.

3. Encapsulamiento:

- Los objetos deben encapsular tanto **datos** como **comportamiento**. Esto significa que el acceso a los datos solo se debe hacer a través de métodos definidos dentro del objeto.

4. Tipos o Clases:

- Las BDOO deben soportar una estructura organizada en **tipos** o **clases**, lo que permite organizar objetos en categorías con propiedades y comportamientos comunes.

5. Jerarquía:

- La **herencia** debe ser soportada, permitiendo que las clases hereden atributos y comportamientos de otras clases (superclases), facilitando la reutilización de código y la organización jerárquica de datos.

6. Ligadura Tardía:

- Se debe permitir la **sobreescripción de métodos** y el uso de **ligadura tardía**, lo que implica que el comportamiento de un objeto puede cambiar en tiempo de ejecución, lo que se utiliza en la programación orientada a objetos.

7. Completitud:

- El sistema de gestión de base de datos debe ser capaz de **expresar todas las funciones calculables** y no debe limitarse solo a operaciones básicas de consulta y almacenamiento.

8. Extensibilidad:

- El sistema debe permitir **agregar nuevos tipos** de objetos según sea necesario, permitiendo la expansión del modelo de datos sin grandes modificaciones al sistema.

Ventajas de los ODBMS (Sistemas de Gestión de Bases de Datos Orientadas a Objetos)

1. Almacenamiento directo de objetos:

- Los **objetos** definidos en las aplicaciones pueden ser almacenados directamente en la base de datos sin necesidad de transformaciones complejas, lo que facilita el trabajo con datos complejos.

2. No conversión de tipos:

- No se necesita conversión entre los tipos de datos en la aplicación y en la base de datos. Las clases definidas por el usuario en la aplicación pueden usarse directamente como tipos en la base de datos.

3. Integración con lenguajes orientados a objetos:

- El lenguaje de consulta y manipulación de la base de datos puede estar completamente integrado con el lenguaje de programación orientado a objetos utilizado en la aplicación, evitando tener dos representaciones de los objetos (como ocurre en las bases de datos relacionales).

Frameworks

Un **framework** es una estructura de software que proporciona funcionalidades reutilizables y personalizables para un propósito específico. En términos generales:

1. Conjunto extensible de clases e interfaces:

- El framework incluye clases e interfaces que definen las interacciones y servicios básicos del subsistema que está modelando.

2. Clases abstractas:

- Un framework generalmente contiene clases abstractas que establecen la estructura del sistema y pueden tener métodos tanto abstractos como concretos.

3. Extensibilidad:

- Los usuarios del framework pueden extenderlo creando subclases de las clases proporcionadas, adaptando los servicios del framework a sus necesidades.

4. Principio de Hollywood:

- En un framework, se aplica el principio "No nos llames, nosotros te llamaremos", lo que significa que el flujo de control está determinado por el framework y no por el usuario de este.

5. Reutilización de código:

- Los frameworks permiten una **gran reutilización** de código, proporcionando una infraestructura común para construir aplicaciones y sistemas de manera más rápida y eficiente.

Framework de Persistencia

Un **framework de persistencia** se centra en facilitar la interacción con mecanismos de almacenamiento persistente (como bases de datos) al ofrecer un conjunto de funciones y herramientas para:

1. **Almacenamiento y recuperación de objetos:**

- Permite almacenar y recuperar objetos en mecanismos de almacenamiento persistente, como bases de datos, sin que el programador tenga que gestionar manualmente la persistencia de los objetos.

2. **Gestión de transacciones:**

- Proporciona funciones para gestionar las transacciones, como **commit** (confirmar cambios) y **rollback** (deshacer cambios), asegurando la consistencia de los datos durante las operaciones de almacenamiento.

Este tipo de framework abstrae muchos de los detalles técnicos y operacionales asociados con el almacenamiento de objetos, lo que facilita la creación y mantenimiento de aplicaciones que interactúan con bases de datos.

7. Arquitectura

Arquitectura:

Es el planteo estratégico para satisfacer los requerimientos de un producto de software, definiendo cómo estructurar y organizar sus elementos. Incluye decisiones clave que guían el proyecto, garantizando eficiencia, adaptabilidad y calidad.

Arquitectura de software:

Es la estructura y relación de componentes del sistema, orientada a su tecnología de aplicación. Define lineamientos que todos los equipos deben seguir, especificando:

- **Estructura:** cómo se organizan los componentes.
- **Comunicación:** cómo interactúan los componentes.
- **Requerimientos no funcionales:** direcciona atributos como seguridad, desempeño, disponibilidad, etc.
- **Abstracción:** provee una visión simplificada del sistema.

Arquitecto:

Es el responsable en las etapas de requerimientos, análisis y diseño. Actúa como líder técnico, puente entre equipos, y gestor de riesgos.

Debe tener 4 habilidades:

- **Enlace:** Se relacionan con la gerencia, justificando diseños, decisiones y costos. Sirve para traducir y explicar diferentes terminologías entre las partes interesadas.
- **Ingeniería de Software:** Sus diseños deben estar debidamente documentados y comunicados y sus planes deben ser explícitos y justificados. Deben comprender el impacto posterior de sus decisiones, trabajando apropiadamente con la prueba de la aplicación, la documentación y equipos de lanzamiento
- **Conocimiento Tecnológico:** Conocimiento de los dominios tecnológicos que son relevantes para los tipos de aplicaciones en las que trabajan. Son influyentes en la evaluación y elección de componentes de terceros y tecnologías.

- **Gestión de Riesgos:** Desarrollan estrategias de mitigación de riesgos, y las comunican a los equipos de ingeniería pertinentes. Intentan asegurarse de que, si ocurren desastres, no sea inesperado.

Sus principales roles son:

- Revisar y negociar requerimientos: es necesario conocer la organización, la cultura de dicha organización, las consideraciones del negocio, qué habilidades tienen las personas que van a utilizar el producto. (relación con ux) Cuando no se tienen en cuenta estos aspectos, es cuando los productos no cumplen las expectativas de los clientes y fracasan.
- Documentar la arquitectura mediante vistas (UML) y decisiones técnicas.
- Comunicar la arquitectura a partes interesadas y priorizar necesidades.
- Asegurar que se respeten los lineamientos arquitectónicos.

Requerimientos no funcionales

Son cualidades o atributos del sistema que condicionan su diseño, como desempeño, seguridad, disponibilidad y mantenibilidad. Están ligados a la arquitectura y deben priorizarse según su **criticidad**:

- **Desempeño:** optimiza tiempo de respuesta y rendimiento.
- **Seguridad:** protege datos con capas y validaciones.
- **Disponibilidad:** utiliza redundancia para evitar caídas.
- **Mantenibilidad:** fomenta modularidad, bajo acoplamiento y alta cohesión.

Elección del modelo arquitectónico

Basada en patrones o estilos arquitectónicos, según los requerimientos no funcionales. Ejemplos:

- **Performance:** componentes de granularidad gruesa para reducir comunicaciones (es decir MENOS subsistemas que tienen MÁS componentes y que hacen MUCHAS cosas) para reducir componentes de comunicación.
 - Rendimiento: Trabajo realizado en un tiempo específico, medido en transacciones por segundo (tps) o mensajes procesados por segundo (mps). El rendimiento máximo es crítico en situaciones como eventos en aplicaciones de apuestas para evitar pérdidas económicas.
 - Tiempo de Respuesta: Duración desde una entrada hasta obtener una respuesta, diferenciando entre promedios y tiempos garantizados. Ejemplo: aplicaciones en tiendas necesitan respuestas rápidas para eficiencia.
 - Plazos: Cumplimiento de tareas en tiempos específicos, como en sistemas batch. Ejemplo: un sistema de pagos debe procesar transacciones en un día para evitar problemas.
- **Seguridad:** estructura en capas para proteger subsistemas sensibles.
 - Autenticación: Verificar la identidad de usuarios y aplicaciones.
 - Autorización: Controlar derechos de acceso según el rol (por ejemplo, solo lectura o lectura/escritura).
 - Encriptación: Proteger mensajes durante su envío y recepción mediante cifrado.
 - Integridad: Garantizar que los mensajes no sean modificados durante su tránsito.

- **No repudio:** Proporcionar pruebas tanto de entrega como de origen, evitando que las partes involucradas nieguen su participación.
 - **Protección:** Requiere centralizar operaciones relacionadas en pocos subsistemas para reducir costos y facilitar la validación. Define qué puede o no hacerse en el sistema.
 - **Disponibilidad:** Depende de la confiabilidad y el tiempo medio entre fallos. Si una aplicación no está disponible cuando se necesita, es poco probable que cumpla con sus requerimientos funcionales
 - **Redundancia:** Permite continuar operaciones incluso con fallas, aunque con posible degradación del rendimiento.
 - **Recuperabilidad:** Restablece datos y rendimiento tras fallos. Métrica clave: tiempo medio de recuperación.
 - **Mantenibilidad:** Mide qué tan fácil es mantener la aplicación una vez que ha sido implementada. Facilita el soporte y las actualizaciones. (relación con evolución de software)
 - **Modularidad:** Uso de componentes autocontenidos con alta cohesión y bajo acoplamiento.
 - **Herramientas de diagnóstico:** Por ejemplo, registros de errores que identifican causas de fallos.
 - Permite realizar cambios sin afectar el funcionamiento global del sistema, asegurando la longevidad del software.
-

Conflictos arquitectónicos

Los conflictos arquitectónicos ocurren cuando decisiones en el diseño del software priorizan ciertos aspectos, comprometiendo otros. Ejemplos y soluciones:

1. Granularidad

- **Problema:**
 - Granularidad gruesa mejora la **performance**, pero reduce la **mantenibilidad**.
- **Solución:**
 - Implementar granularidad híbrida:
 - Componentes pequeños para mantener la mantenibilidad.
 - Módulos grandes cuando se requiera alta performance.
- **Ejemplo:**
 - **Problema:** Un módulo único para cuentas de ahorro y tarjetas de crédito mejora **performance**, pero dificulta actualizaciones (baja mantenibilidad).
 - **Solución:** Dividir en módulos especializados para cada función y mantener uno principal para casos críticos.

2. Datos Redundantes

- **Problema:**
 - Mejoran la **disponibilidad**, pero complican la **seguridad** y afectan la **performance**.
- **Solución:**
 - Sincronización automática de datos.

- Refuerzo en seguridad con cifrado y autenticación.
- Optimización de validaciones para minimizar impacto en la performance.

- **Ejemplo:**

- **Problema:** Guardar inventario en dos bases mejora **disponibilidad**, pero genera inconsistencias y reduce seguridad y **performance**.
- **Solución:** Sincronización automática entre bases y validación optimizada con cifrado.

3. Localización de aspectos de seguridad

- **Problema:**

- Aumentar comunicación para gestionar **seguridad** degrada la **performance**.

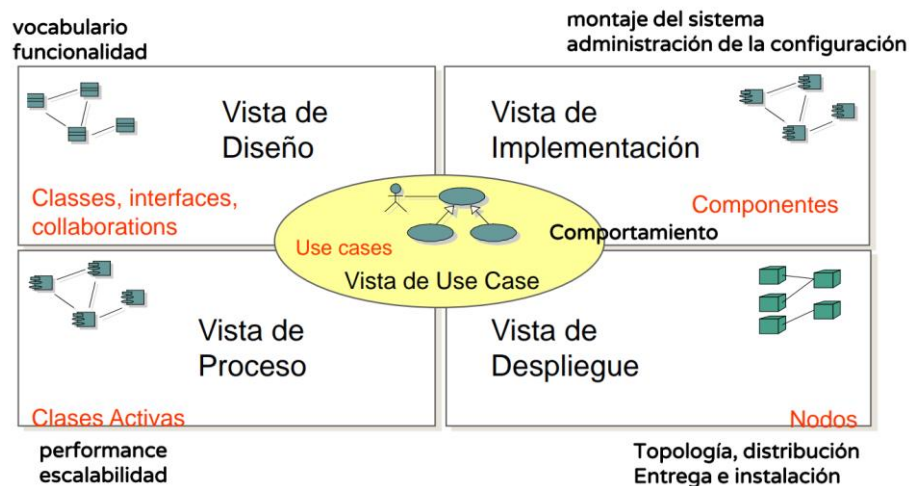
- **Solución:**

- Usar seguridad distribuida y centralizada:
 - Sistemas de gestión de identidad centralizados.
 - Reducir comunicación excesiva para mantener balance entre seguridad y rendimiento.

- **Ejemplo:**

- **Problema:** Cada módulo genera sus claves, lo que mejora seguridad, pero afecta **performance** por exceso de comunicación.
- **Solución:** Centralizar la gestión de claves en un único servicio

VISTAS ARQUITECTONICAS (4+1)



El modelo de **vistas 4+1** es un enfoque para describir la arquitectura de un sistema de software desde diferentes perspectivas, unificadas por los casos de uso. Cada vista tiene un propósito específico y emplea diagramas para representar su parte estática y dinámica

A partir de la Vista de Casos de Uso (vista de la funcionalidad) se van a construir las demás vistas. Con esto se aplica la característica de **PUD “Conducido por casos de uso”** porque de cada caso de uso, se va derivando la traza con los otros modelos que nos interesa mostrar y evolucionar.

1. Vista de Funcionalidad (Vista de Casos de Uso)

- **Propósito:** Capturar y visualizar los principales casos de uso del sistema, mostrando cómo los objetos y procesos interactúan para cumplir con los requisitos funcionales.
 - **Representaciones:**
 - **Estática:** Diagrama de casos de uso.
 - **Dinámica:** Diagrama de secuencia o de comunicaciones.
 - **Importancia:** Actúa como base para las demás vistas y representa los requerimientos funcionales clave.
-

2. Vista de Diseño

- **Propósito:** Mostrar cómo las clases, subsistemas e interfaces trabajan juntos para implementar los casos de uso.
 - **Enfoque:** Distribuir la funcionalidad y definir la estructura del sistema.
 - **Representaciones:**
 - **Estática:** Diagrama de componentes.
 - **Dinámica:** Diagrama de secuencia.
-

3. Vista de Procesos

- **Propósito:** Enfocarse en los aspectos dinámicos del sistema, como procesos concurrentes y su comunicación.
 - **Uso:** Resaltar procesos automáticos y características como rendimiento y concurrencia.
 - **Representaciones:**
 - **Estática:** Diagrama de actividad.
 - **Dinámica:** Diagrama de secuencia.
-

4. Vista de Implementación

- **Propósito:** Representar la arquitectura desde el punto de vista del programador, incluyendo la administración de los artefactos de software.
 - **Enfoque:** Cómo se organizan y gestionan los componentes del software.
 - **Representaciones:**
 - **Estática:** Diagrama de paquetes o componentes.
 - **Dinámica:** Diagrama de secuencia.
-

5. Vista de Despliegue

- **Propósito:** Mostrar cómo los componentes del sistema se distribuyen en la infraestructura física (hardware y nodos).
- **Enfoque:** Conexiones físicas y topología de componentes en la capa física.
- **Representaciones:**

- **Estática:** Diagrama de despliegue.
- **Dinámica:** Diagrama de secuencia.

VISTA ARQUITECTONICA	ESTÁTICA	DINÁMICA
Funcionalidad	Diagrama de CU Sale del WF de requerimientos	Diagrama de secuencia y/o comunicación Se construye en el WF de análisis
Diseño	Diagrama de componentes y/o clases Se construye en el WF de diseño	Diagrama de secuencia Se construye en el WF de diseño
Proceso	Diagrama de componentes Se construye en el WF de diseño	Diagrama de secuencia Se construye en el WF de diseño
Despliegue	Diagrama de despliegue Se construye en el WF de diseño	Diagrama de secuencia Se construye en el WF de diseño
Implementación	Diagrama de componentes Se construye en el WF de diseño	Diagrama de secuencia Se construye en el WF de diseño

Documentación de la Arquitectura

Importancia de documentar la arquitectura

1. Comunicación y comprensión:

- Facilita la comunicación entre partes interesadas.
- Ayuda a entender el diseño incluso después de un tiempo.
- Sirve como herramienta de aprendizaje para nuevos integrantes.

2. Análisis y evaluación:

- Permite analizar el diseño, evaluarlo y medir métricas como acoplamiento y cohesión.
- Garantiza la calidad del software.

3. Registro y justificación:

- Documenta decisiones importantes, como elección de frameworks o patrones.
- Justifica la prioridad y el enfoque de los requisitos no funcionales (RNF).

4. Reutilización y evolución:

- Facilita la reutilización del diseño en otros proyectos.
- Ayuda a reflejar la evolución del sistema y comparar arquitecturas.

5. Planificación y verificación:

- Garantiza que la implementación siga principios establecidos.
- Gestiona cambios de manera controlada.

Desafíos al documentar arquitectura

1. Falta de estándares universales:

- No existe un estándar único para todos los proyectos.

2. Complejidad:

- Las arquitecturas complejas son difíciles de documentar de forma comprensible.

3. Múltiples vistas:

- Documentar todas las vistas relevantes puede ser costoso y demandar recursos.

4. Evolución del diseño:

- Mantener la documentación actualizada es complicado con plazos ajustados. Esta tarea a menudo se pasa por alto, lo que puede llevar a problemas en etapas posteriores del desarrollo.

Qué documentar

- Depende de factores como:
 - **Complejidad de la arquitectura:** Un sistema simple podría requerir solo diagramas básicos.
 - **Vida útil de la aplicación:** Aplicaciones a largo plazo necesitan mayor documentación.
 - **Tamaño del equipo y necesidades de comunicación:**
 - Equipos pequeños: menor necesidad de documentación.
 - Equipos grandes o distribuidos: documentación esencial para aspectos clave como:
 - Interfaces, restricciones, escenarios de prueba, dependencias y servicios externos.

Sistemas Distribuidos

Un **sistema distribuido** es una colección de computadoras independientes que cooperan para brindar al usuario la apariencia de un sistema único y coherente. A diferencia de los sistemas centralizados, el procesamiento de información se distribuye entre varias máquinas.



Transparencia

Un sistema distribuido debe ser transparente. Esto significa que los usuarios ven el sistema como un solo sistema cuyo comportamiento no resulta afectado por la forma en que se distribuye el sistema. En la práctica, es imposible hacer un sistema por completo transparente (demoras en la red, fallas del nodo remoto, etc.).

Características

1. **Compartición de recursos:** Permite usar hardware y software entre varias computadoras. Esto permite aprovechar recursos que no disponemos físicamente y sí están en otro lugar, lo cual aporta a la capacidad de procesamiento.
2. **Apertura:** Basados en estándares para interoperabilidad.
3. **Concurrencia:** Procesos simultáneos en distintas máquinas.
4. **Tolerancia a fallas:** Funciona degradado ante errores; solo falla completamente si la red lo hace.
5. **Escalabilidad:** Capacidad de crecer en tamaño, distribución y manejabilidad.

Desventajas

1. **Complejidad:** Difíciles de diseñar, probar y mantener.
2. **Seguridad:** Más puntos vulnerables.
3. **Manejabilidad:** Difícil administrar sistemas heterogéneos.
4. **Impredecibilidad:** Rendimiento variable por red y carga.

Amenazas

1. **Intercepción:** Espionaje de datos.
2. **Interrupción:** Saturación de servicios.
3. **Modificación:** Alteración de datos o servicios.
4. **Fabricación:** Creación de información falsa para conseguir ciertos privilegios.

Falacias de los Sistemas Distribuidos

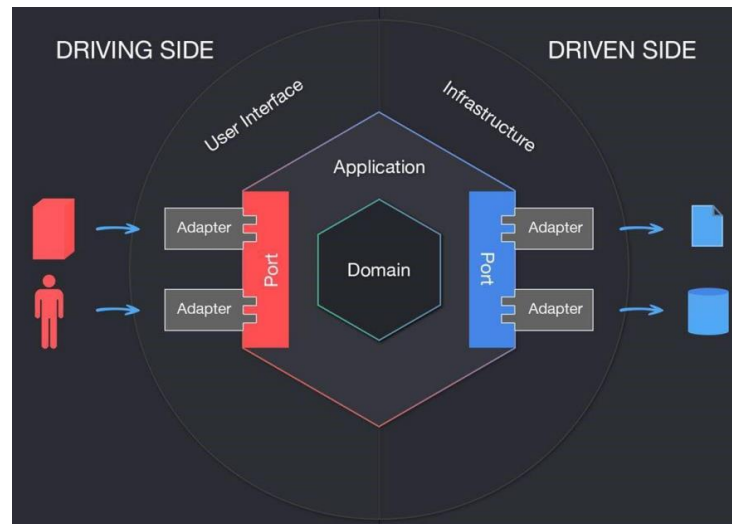
1. **La red está siempre disponible**
2. **Latencia cero:** Siempre hay retrasos en la comunicación.
3. **Ancho de banda infinito**
4. **La red es segura:** Las redes son vulnerables a ataques y requieren medidas de protección.
5. **La topología no cambia:** Las redes pueden ser dinámicas (por ejemplo, en la nube o móviles).
6. **Un único administrador:** Muchos sistemas no tienen un control centralizado.
7. **Costo de transporte cero:** Transferir datos tiene costos y afecta el rendimiento.
8. **La red es homogénea:** Los nodos suelen ser heterogéneos en capacidades y recursos.

Arquitectura Hexagonal

Es un diseño que separa la lógica de negocio central de los detalles externos mediante puertos (interfaces) y adaptadores (implementaciones). Representada como un hexágono, prioriza la independencia tecnológica y la mantenibilidad.

Componentes principales:

1. **Lógica de negocio (núcleo):** Contiene las reglas y algoritmos centrales, definiendo interfaces (puertos).
2. **Puertos:** Interfaces que conectan el núcleo con sistemas externos, independientes de tecnologías específicas.
3. **Adaptadores:** Implementan los puertos y actúan como puente con tecnologías externas (APIs, bases de datos, etc.).
4. **Sistemas externos:** Interactúan con la aplicación a través de los adaptadores y puertos.



Funcionamiento:

- **Lado conductor:** Adaptadores que inician la interacción (ej., controladores REST).
- **Lado conducido:** Adaptadores que responden a solicitudes del núcleo (ej., acceso a base de datos).

Los **puertos** actúan como contratos para mantener la independencia entre el núcleo y las implementaciones externas.

Beneficios:

- **Acoplamiento bajo:** El núcleo es independiente de las tecnologías externas.
- **Alta cohesión:** La lógica de negocio se mantiene enfocada.
- **Pruebas mejoradas:** Fácil de testear por su modularidad.
- **Mantenimiento sencillo:** Cambios en adaptadores no afectan el núcleo.
- **Escalabilidad:** Integra múltiples sistemas y soporta alto volumen de datos.
- **Reusabilidad:** Adaptadores intercambiables sin alterar el núcleo.

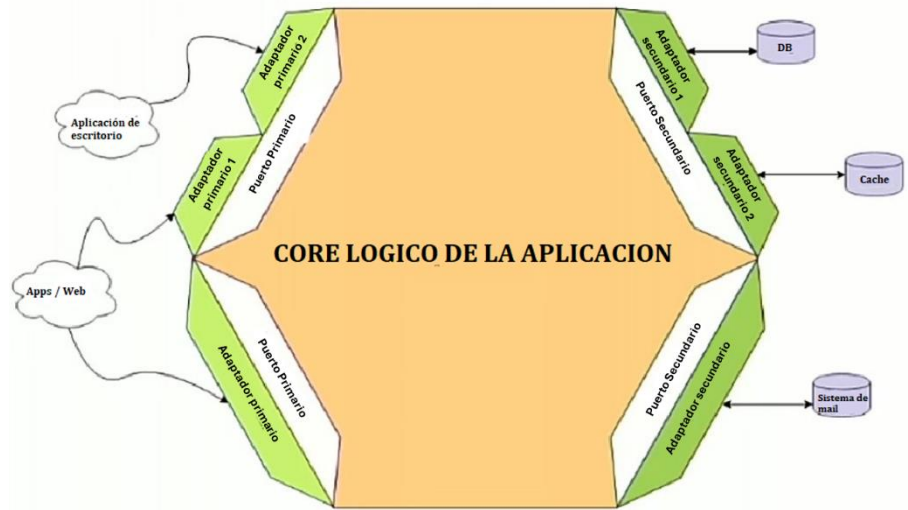
Comparación Layered vs Hexagonal:

- **Layered (en capas):**
 - Centrada en la base de datos.
 - Fuerte acoplamiento entre capas y difícil de mantener.
 - Menos flexible ante cambios tecnológicos.
- **Hexagonal:**
 - Centrada en la lógica de negocio.
 - Bajo acoplamiento y fácil integración con nuevas tecnologías.
 - Adaptable y escalable.

Inversión de dependencia:

- **Driving Side:** Los adaptadores dependen de los puertos (interfaces del núcleo).
- **Driven Side:** El núcleo utiliza los puertos, mientras que los adaptadores implementan su lógica.

Este principio asegura que la lógica de negocio sea independiente de los detalles técnicos, favoreciendo modularidad y flexibilidad.



Aplicaciones recomendadas:

- Integración con múltiples sistemas.
- Grandes volúmenes de datos.
- Pruebas exhaustivas.
- Escalabilidad horizontal.

Arquitectura Monolítica vs Microservicios

Monolítica:

Una aplicación empaquetada como una sola unidad donde todos los módulos están interconectados.

Ventajas:

1. **Fácil desarrollo y prueba:** Ideal para proyectos iniciales por su simplicidad.
2. **Despliegue sencillo:** Se implementa como una sola unidad.
3. **Rendimiento óptimo:** Sin comunicación distribuida, todo el procesamiento es local.
4. **Menos puntos de fallo:** No depende de múltiples servicios externos.
5. **Escalabilidad básica:** Puede escalarse usando balanceadores de carga.

Desventajas:

1. **Complejidad creciente:** Difícil de manejar en aplicaciones grandes.
 2. **Fallas críticas:** Un error puede detener todo el sistema.
 3. **Mantenimiento complicado:** Cambios menores requieren nuevos despliegues completos.
 4. **Escalabilidad limitada:** Difícil escalar módulos específicos.
 5. **Dependencia tecnológica:** Difícil migrar a nuevas tecnologías.
-

Microservicios:

Arquitectura basada en componentes pequeños e independientes, cada uno enfocado en una tarea específica y con comunicación vía APIs. Los servicios se conectan a través de interfaces (una API)

Ventajas:

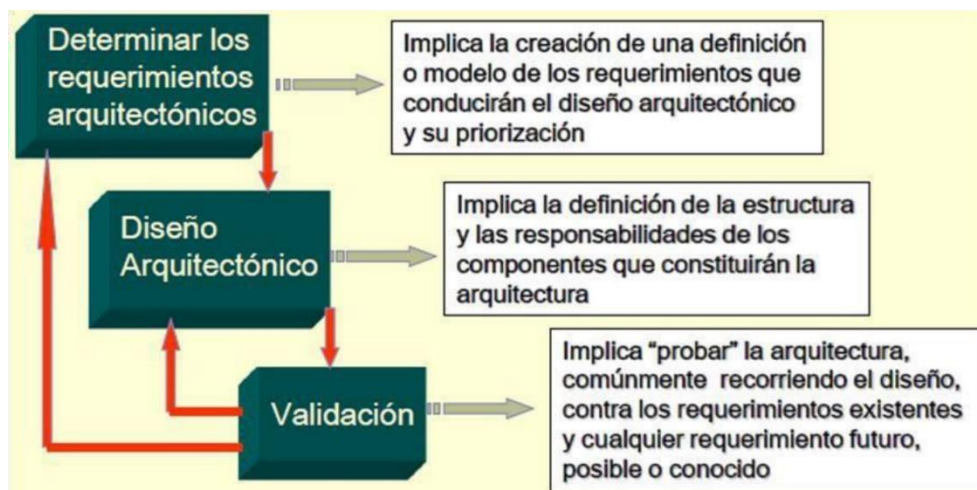
1. **Escalabilidad granular:** Se pueden escalar servicios específicos según la demanda.
2. **Desarrollo independiente:** Equipos pueden trabajar en diferentes servicios simultáneamente.
3. **Aislamiento de fallas:** Si un servicio falla, el resto sigue funcionando.
4. **Flexibilidad tecnológica:** Cada servicio puede usar distintas tecnologías y bases de datos.
5. **Actualizaciones frecuentes:** Permite iteraciones y despliegues rápidos.
6. **Reusabilidad:** Los servicios pueden ser reutilizados en múltiples aplicaciones.

Desventajas:

1. **Complejidad del sistema:** Difícil coordinar y mantener sistemas distribuidos.
2. **Pruebas distribuidas:** Requiere mayor esfuerzo para simular el entorno real.
3. **Comunicación inter-servicios:** Incrementa la latencia y la necesidad de soluciones robustas.
4. **Gestión de transacciones:** Difícil implementar transacciones en múltiples servicios.
5. **Consumo de recursos:** Cada servicio independiente aumenta el uso de memoria y CPU.

8. Diseño Arquitectónico

El **proceso de diseño de arquitectura de software** según [Ian Gorton](#) se organiza en tres pasos fundamentales:



1. Determinar los Requerimientos Arquitectónicos

- Se parte del modelo de requerimientos (**requerimientos funcionales y no funcionales**) ya definidos.
- Se **priorizan** los requerimientos en tres categorías:
 - **Alta:** Requerimientos esenciales que guían el diseño desde la 1er iteración.
 - **Media:** Requerimientos que pueden implementarse en fases posteriores.
 - **Baja:** Requerimientos deseables, pero no esenciales.

- La priorización se adapta a **iteraciones**, considerando qué se debe cumplir desde el primer momento y qué puede postergarse. Si el ciclo de vida fuese en cascada no tendría sentido la priorización porque se hace todo junto

2. Diseño Arquitectónico

La salida es un conjunto de vistas arquitectónicas que capturan el diseño de la arquitectura y un documento de diseño que explica el diseño, las razones clave de algunas de las principales decisiones de diseño, e identifica los riesgos involucrados.

1. **Elegir el Framework:** Se seleccionan **patrones arquitectónicos** adecuados para abordar los **atributos de calidad** y minimizar riesgos.
2. **Distribuir Componentes:** Se diseña la estructura de componentes:
 - Minimizar dependencias entre ellos.
 - Cada componente debe tener responsabilidades **altamente cohesivas**.
 - Reducir llamadas entre componentes distribuidos.
 - Descomponer los componentes jerárquicamente para mejorar su organización.

3. Validación

- Se busca aumentar la **confianza** de que la arquitectura cumplirá con los requerimientos dentro de los límites de tiempo y presupuesto.
- Se utilizan dos técnicas principales:
 - **Escenarios de prueba:** Evaluación manual de la arquitectura con distintos **escenarios de prueba** para detectar defectos.
 - **Prototipado:** Los prototipos son versiones mínimas o reducidas de la aplicación deseada, creada específicamente para probar algunos aspectos del diseño mal entendido con alto riesgo. Se usan para validar tecnologías y la viabilidad de la arquitectura mediante:
 - **Pruebas de concepto:** Es probar si todo lo que has planeado, como el diseño y los componentes, puede satisfacer los requisitos del proyecto
 - **Pruebas de tecnología:** Esto es probar las herramientas y tecnologías que vas a usar para construir la aplicación (middleware, aplicaciones integradas, bibliotecas, etc.)

Estas fases son iterativas, con el objetivo de asegurar que la arquitectura diseñada sea robusta y cumpla con las expectativas del sistema final.

Patrones arquitectónicos

Son prácticas abstractas de diseño que se han probado en diversos sistemas y entornos. Describen cuándo usarlos y sus pros y contras.

Framework arquitectónico: Es el conjunto de patrones arquitectónicos utilizados para estructurar una aplicación. No se opone a la idea de tener un estilo arquitectónico predominante, pero agrupa varios patrones que una aplicación puede emplear, como Layered, N-Tier o Publish & Subscribe.

Patrones vs. Estilos arquitectónicos:

Diferencia con patrones de diseño

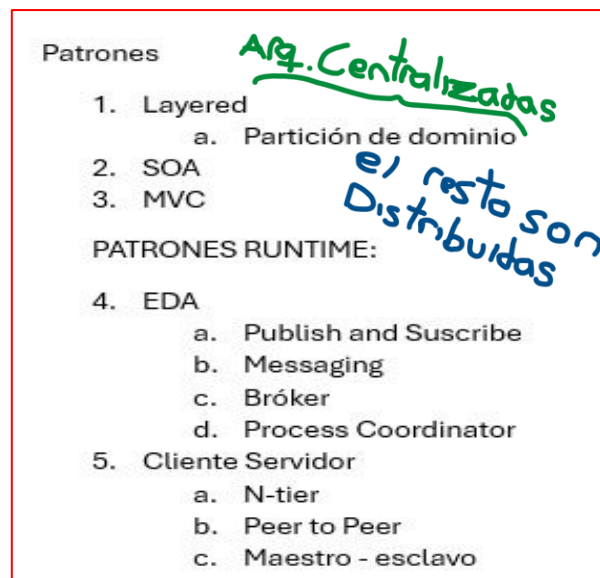
Los **patrones arquitectónicos** son soluciones generales y reutilizables para organizar la estructura global de un sistema de software, definiendo cómo los componentes principales interactúan entre sí (por ejemplo, monolítica, microservicios, hexagonal). En cambio, los **patrones de diseño** son soluciones específicas y reutilizables para problemas recurrentes en el diseño detallado de software a nivel de componentes o clases, enfocándose en cómo se resuelven tareas concretas dentro de la arquitectura (como Singleton u Observer). Mientras los patrones arquitectónicos abordan el **qué** y el **cómo** del diseño global, los de diseño se centran en el **cómo** dentro del nivel más granular.

- **Patrones arquitectónicos:** Son más pequeños en escala y varios pueden coexistir en un diseño.
- **Estilos arquitectónicos:** Es el patrón principal que estructura y determina el comportamiento de un sistema. Un sistema generalmente tiene un solo estilo arquitectónico dominante.

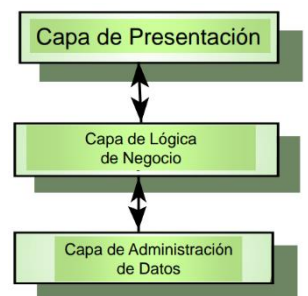
Platónicos vs. Embebidos:

- **Platónicos:** Son patrones ideales que se encuentran en libros, pero rara vez se implementan tal cual en el código.
- **Embebidos:** Son los patrones reales que se ven en sistemas prácticos, donde a menudo se violan las reglas estrictas de los patrones platónicos, lo que implica compensaciones para ajustarse a la realidad.

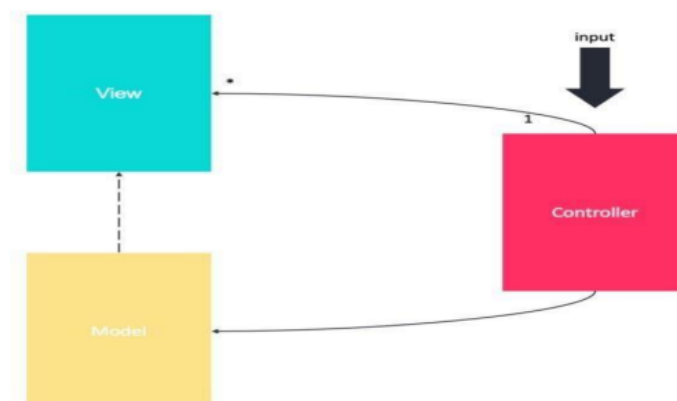
Patrones



- **Layered:** Organiza el sistema en capas, donde cada capa solo puede usar la capa inmediatamente inferior. Cada capa ofrece servicios a la capa superior. (principio SOLID SoC presente)
 - **Se usa cuando:**
 - Al construir nuevas funcionalidades sobre sistemas existentes.
 - Cuando el desarrollo se distribuye entre varios equipos.
 - En sistemas que requieren una estructura multinivel.
 - **Ventajas:**
 - Fácil de desarrollar, probar y mantener
 - Permite sustituir capas completas sin afectar la interfaz.
 - Aumenta la confiabilidad al permitir redundancia en cada capa.
 - **Desventajas:**
 - Difícil de **escalar** por ser monolítica
 - El rendimiento puede verse afectado debido a los múltiples niveles de procesamiento.
- ❖ **Partición de Dominio:** Divide el sistema según necesidades del negocio, no en capas técnicas. Cada subdominio refleja una parte específica del negocio, como gestión de clientes o servicio de delivery.
 - **Ventajas:**
 - Refleja mejor el funcionamiento del negocio
 - Compatible con Arq. modulares y de microservicios
 - Facilita la migración a arquitecturas distribuidas
 - **Desventaja:**



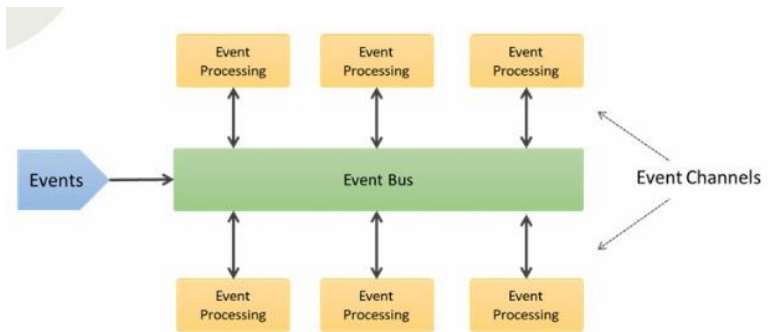
- La personalización del código puede dispersarse en varios lugares
- **SOA – Arquitectura Orientada a Servicios**
 - **División en Servicios:** Separa aplicaciones monolíticas en servicios independientes, cada uno con una función específica. Estos servicios se comunican a través de API definidas, permitiendo desarrollo y gestión independientes.
 - **Ventajas:**
 - Bajo Acoplamiento: Los servicios están débilmente acoplados, lo que facilita la reutilización y la integración.
 - Testeabilidad: Cada servicio puede probarse individualmente.
 - Reutilización y Agilidad: Se pueden crear nuevos servicios a partir de los existentes.
 - Escalabilidad: Permite agregar instancias y balancear la carga.
 - Interoperabilidad: Compatible con diferentes sistemas operativos y lenguajes.
 - **Desventajas:**
 - Rendimiento: Puede ser menos eficiente.
 - Volumetría: No ideal para grandes volúmenes de datos.
 - Gobierno: Requiere una gestión estricta de servicios.
 - Puntos de Falla: Más servicios implican más posibles fallas.
- **Modelo-Vista-Controlador (MVC)**
 - Divide una aplicación en tres componentes principales:
 - Modelo: Maneja la lógica del negocio y el estado de la aplicación. Gestiona los datos, incluyendo la lectura, escritura, y validación, y puede encargarse de la red y la persistencia de los datos.
 - Vista: Presenta los datos al usuario y gestiona la interacción con él.
 - Controlador: Conecta el modelo y la vista, coordinando la interacción entre ambos.
 - Este patrón busca mejorar la modularidad, flexibilidad, testeabilidad y mantenibilidad del código.
 - **Ventajas:**
 - Los datos y su representación visual pueden cambiar de manera independiente.
 - Soporta múltiples formas de visualizar los mismos datos, actualizando todas las vistas cuando hay cambios.
 - **Desventaja:**
 - Puede añadir complejidad y código innecesario cuando el sistema es sencillo



PATRONES RUNTIME (VISTA DE EJECUCIÓN)

- **Event Driven Architecture (EDA)**

Modelo de arquitectura asíncrona y distribuida diseñado para aplicaciones altamente escalables. En EDA, las aplicaciones generan eventos (cambios importantes en su estado), y otros componentes reaccionan a esos eventos, creando un flujo dinámico que mejora la escalabilidad y flexibilidad del sistema.



- **Ventajas:**

- Escalabilidad: Los consumidores de eventos pueden escalar de forma independiente, minimizando el acoplamiento entre componentes.
- Despliegue sencillo: Al estar desacoplados, los componentes se despliegan sin preocuparse por dependencias.
- Flexibilidad: Permite que los componentes se adapten rápidamente a cambios, al estar cada uno especializado en una tarea.

- **Desventajas:**

- Testeabilidad: Dificultad para probar debido a la naturaleza distribuida y asíncrona.
- Desarrollo complicado: Implementar soluciones asíncronas y manejar errores es más complejo, lo que puede generar inconsistencias en el sistema.

VARIANTES DEL EDA:

- **Publish and Subscribe:** mecanismo en tiempo de ejecución donde componentes independientes publican eventos y otros se suscriben a ellos. Los publicantes no conocen a los suscriptores ni el uso que se dará a los eventos que publican, y los suscriptores dependen únicamente del evento, sin saber quién lo publica

- **Ventajas:**

- **Bajo acoplamiento:** no hay vínculo directo entre publicadores y suscriptores
- **Desacoplamiento:** los componentes que producen los eventos están desacoplados de los que los consumen → Arq. Mantenible y evolucionable



- **Desventajas:**

- **Rendimiento:** Capa de indirección añadida por el bus de eventos. Necesidad de gestionar la comunicación entre múltiples publicantes y suscriptores.

- **Messaging:** Emisores envían mensajes a receptores, donde los mensajes son almacenados en una cola

- **Propiedades:**

- Comunicaciones asíncronas
- Calidad de Servicio configurable
- Bajo acoplamiento

- **Ventajas**

- Bajo acoplamiento: No hay vínculo directo entre emisores y receptores

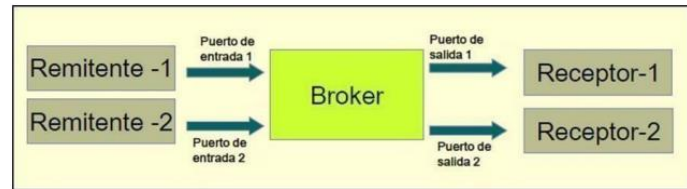
- **Desventaja**

- Dependencia en el formato del mensaje: Los cambios en los formatos de los mensajes de los receptores pueden requerir modificaciones en los emisores. Sin embargo, los mensajes autodescriptivos pueden mitigar esta dependencia.



- **Bróker (agente):**

- Arquitectura Hub-and-spoke: Todos los mensajes van al bróker, que los envía a su destino. Los emisores y receptores están conectados al bróker, que gestiona todas las conexiones.
- Ruteo de mensajes: El bróker decide a dónde enviar cada mensaje que recibe, según las reglas que tenga.
- Transformación de mensajes: El bróker puede cambiar el formato del mensaje para que sea compatible con el receptor, convirtiéndolo del formato que recibe al formato que necesita el destinatario.
- Se usa cuando hay q transformar mucho los mensajes entre diferentes formatos, como con información de sensores o sistemas externos.
- **Ventaja:**
 - Desacoplamiento: El bróker separa el remitente del receptor, permitiendo que ambos usen sus propios formatos de mensaje
 - Centraliza la lógica de transformación, facilitando el mantenimiento y la comprensión.



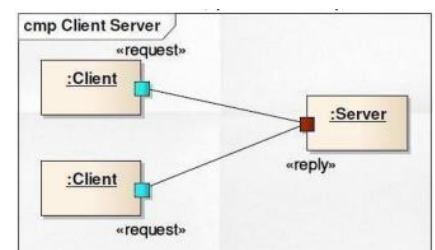
- **Process Coordinator:**

- Encapsula el proceso: El coordinador maneja todos los pasos necesarios para completar un proceso de negocio, como una receta que sigue un orden específico. Recibe una solicitud, llama a los diferentes servidores en el orden correcto y entrega el resultado final.
- **Bajo acoplamiento:** Los servidores no saben qué pasos forman el proceso completo, solo saben qué tareas deben hacer. El coordinador organiza todo.
- Comunicación flexible: El coordinador puede esperar respuestas de los servidores de inmediato (sincrónico) o puede enviar una solicitud y esperar una respuesta en otro momento (asíncrono).
- Se usa para procesos de negocio complejos que quieren coordinar varias tareas
- Ventajas:
 - Fácil de administrar: Todo el proceso se maneja desde un solo lugar, lo que facilita su modificación y monitoreo.
 - Bajo acoplamiento: Los servidores no necesitan conocerse entre sí.
- Desventajas:
 - Posible cuello de botella: Si el coordinador recibe demasiadas solicitudes, puede ralentizarse. Se puede usar redundancia para mejorar esto.



- **Cliente-Servidor**

- **Clientes y Servidores:** Los clientes son las aplicaciones que hacen solicitudes de servicio, y los servidores son las que proporcionan esos servicios. Los clientes saben cómo encontrar a los servidores, pero los servidores no saben nada de los clientes hasta que los contactan.
- **Comunicación Sincrónica:** Los clientes hacen una petición y esperan una respuesta de los servidores.
- **Escalabilidad:** Los servidores pueden ser duplicados para manejar más carga. Esta arquitectura también puede funcionar en una sola computadora.
- Se usa para aplicaciones que necesitan manejar cargas variables o donde se requiere protección centralizada de datos.
- **N-Tier** Utiliza múltiples capas de servidores y clientes. Cada capa tiene una función específica y los pedidos fluyen en una sola dirección a través de las capas

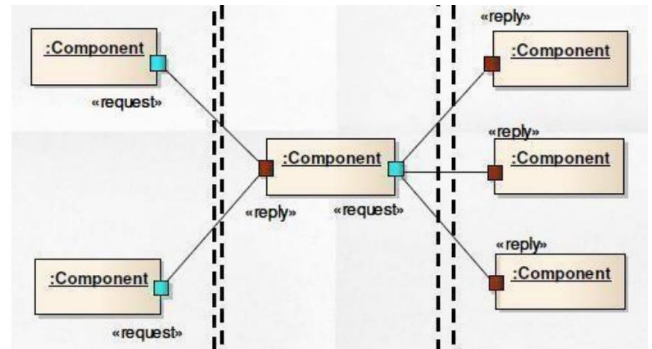


- **Ventajas:**
 - Escalabilidad: Los servidores pueden ser replicados para manejar más usuarios o tareas.
 - Centralización: Los datos y servicios pueden ser centralizados y protegidos.
- **Desventajas:**
 - Dependencia del servidor: Si el servidor tiene problemas, todos los clientes afectados pueden sufrir.

VARIANTES DE CLIENTE-SERVIDOR

• **N-Tier**

- **División en Capas:** La aplicación se divide en capas separadas para la presentación, la lógica de negocio y el manejo de datos. Las capas se comunican de manera sincrónica, enviando solicitudes en una única dirección y esperando respuestas antes de continuar.
- **Flexibilidad en la Distribución:** Las capas pueden estar en la misma máquina o en máquinas diferentes. En aplicaciones web, la capa de presentación suele estar en el navegador, mientras que las capas web y de negocio están en servidores remotos.
- **Variante Embebida:**
 - En esta variante, la capa de presentación actúa como cliente y la capa de acceso a datos como servidor. Las capas intermedias pueden funcionar como clientes y servidores, permitiendo que atiendan peticiones y realicen solicitudes a otras capas.
- Se usa para aplicaciones que deben manejar muchos clientes y solicitudes simultáneas, donde cada solicitud se procesa rápidamente. Es útil para aplicaciones web y sistemas que necesitan alta disponibilidad, requiriendo replicar servidores en cada capa.
- **Ventajas:**
 - Organización Clara: Separación de responsabilidades en diferentes capas.
 - Escalabilidad y Flexibilidad: Las capas pueden ser distribuidas y replicadas según sea necesario.
- **Desventajas:**
 - Complejidad: Puede ser complejo gestionar la comunicación entre múltiples capas.



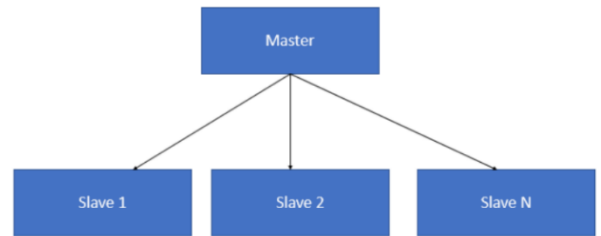
• **Peer to Peer (2p2)**

Permiten que los nodos se comuniquen como pares, sin jerarquías. Cada nodo puede actuar como cliente y servidor, solicitando o proporcionando servicios. Los nodos se conectan dinámicamente a un subconjunto de otros nodos, y las conexiones pueden cambiar en tiempo de ejecución. Estas redes se utilizan para acceder a recursos distribuidos en varios nodos.

- **Ventajas:**
 - **Escalabilidad:** A medida que aumenta el número de nodos, también lo hacen los recursos disponibles y la distribución de la carga.
 - **Tolerancia a fallas:** La distribución de los recursos permite el acceso incluso si algunos nodos fallan.
 - **Descentralización:** No dependen de un servidor central.
 - **Privacidad:** El uso de búsquedas distribuidas aumenta la privacidad de los usuarios.
 - **Equilibrio de carga:** Todos los nodos comparten la misma responsabilidad, distribuyendo la carga de manera equitativa.

- **Desventajas:**

- **Complejidad:** Desarrollar sistemas P2P es complicado, ya que los nodos actúan como clientes y servidores.
- **Control:** Sin un servidor central, es difícil controlar el contenido compartido.
- **Seguridad:** Las redes P2P son vulnerables a ataques, como la propagación de virus o contenido adulterado.
- **Tráfico:** En redes no optimizadas, el tráfico puede saturarse debido a la gran cantidad de solicitudes entre nodos.



- **Maestro – Esclavo**

- Se usan comúnmente en sistemas de tiempo real donde es importante cumplir con los tiempos de procesamiento.
- El proceso “maestro” es por lo general el responsable de la computación, coordinación y comunicación y controla los procesos “esclavos”.
- Los procesos “esclavos” se dedican a acciones específicas, como adquisición de datos de sensores.
- Se usa cuando es posible predecir el procesamiento distribuido que se requiere y cuando se puede asignar fácilmente el procesamiento a los procesadores esclavos

9. Patrones GRASP (General Responsibility Assignment Software Patterns)

Son principios de diseño orientado a objetos que ayudan a asignar responsabilidades a las clases y objetos manera que se logre un sistema robusto, mantenible y reutilizable.

Responsabilidades

Representan las obligaciones de un objeto en cuanto a su comportamiento, no son métodos, pero **se implementan a través de ellos**.

- Responsabilidades del **conocer**:
 - Conocer datos privados encapsulados.
 - Conocer objetos relacionados (atributos o clases vinculadas).
 - Conocer datos que se pueden derivar o calcular (ejemplo: calcularEdad() usando fechaNacimiento).
- Responsabilidades del **hacer**:
 - Realizar acciones propias, como crear objetos, realizar cálculos o responder solicitudes.
 - Delegar acciones a otros objetos.
 - Coordinar actividades entre varios objetos.

2. Patrones

Un patrón de diseño es una solución reutilizable a problemas comunes en el diseño de software, basada en principios probados y nombrada para facilitar su comprensión y comunicación. Los **patrones GRASP** distribuyen responsabilidades entre clases para mejorar la calidad del software, siguiendo principios como bajo acoplamiento y alta cohesión.

Se aplican durante la elaboración de diagramas de interacción y casos de uso, principalmente en las fases de **elaboración y construcción** del workflow de análisis, guiando el diseño hacia sistemas bien estructurados y mantenibles.

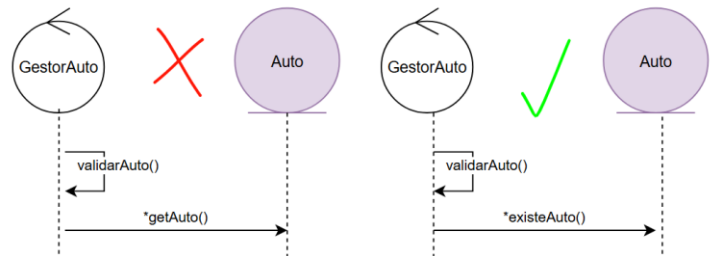
Principales Patrones GRASP

1. Experto en Información (lo hace quien conoce)

- **Problema:** ¿Cómo se deben asignar las responsabilidades en el diseño orientado a objetos?
- **Solución:** Asignar la responsabilidad al objeto que tiene la información necesaria para cumplirla. Tenemos tendencia a sobrecargar a los objetos de control y las clases de entidad

- **Ejemplo:**

- El gestor no debería procesar la existencia de un auto basándose en su nombre; en su lugar, delega esta tarea al objeto Auto, que conoce su propio nombre.



- **Beneficios:**

- Mantiene el **encapsulamiento**: las clases usan su propia información.
- Reduce el **acoplamiento**: menos dependencias entre clases.
- Incrementa la **cohesión**: cada clase realiza tareas relacionadas con su propósito.

2. Creador

- **Problema:** ¿Qué clase debería tener la responsabilidad de crear instancias de otra clase?
- **Solución:** Asignar a la clase B la responsabilidad de crear la clase A cuando se cumple uno o más de los casos siguientes:

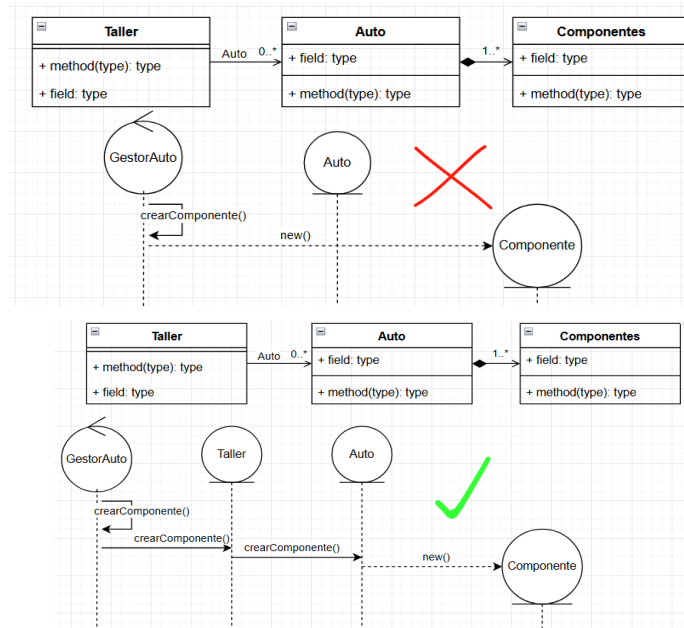
1. La clase B está formada por A
2. La clase B está compuesta por la clase A.
3. La clase B agrega/contiene a la clase A.
4. La clase B utiliza a A.
5. La clase B tiene los datos necesarios para inicializar A.

- **Ejemplo:**

- Si un Carro contiene Ruedas, el Carro es el encargado de crear sus Ruedas.

- **Beneficios:**

- Mantiene el **bajo acoplamiento**: evita que otras clases interfieran innecesariamente.
- Facilita la **reutilización** al limitar dependencias entre clases.

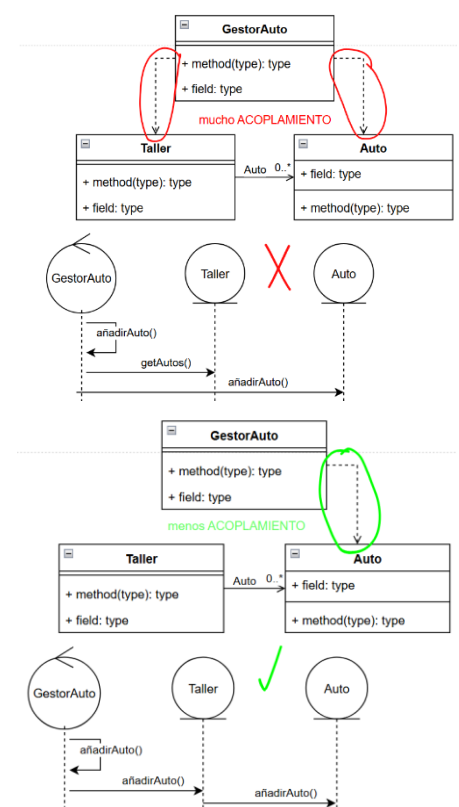


3. Alta Cohesión

- **Problema:** ¿Cómo evitar clases con demasiadas responsabilidades o que asumen tareas no relacionadas? Una clase con baja cohesión hace muchas cosas no relacionadas, o hace demasiado trabajo.
- **Solución:** Asignar responsabilidades para que las clases mantengan un enfoque claro y colaboren cuando sea necesario.
- **Definición:**
 - Una clase es cohesiva cuando sus responsabilidades están estrechamente relacionadas y realiza solo las tareas necesarias para su propósito.
- **Ejemplo:**
 - Una clase de Usuario debería gestionar solo datos y operaciones relacionadas con el usuario, delegando tareas externas (como guardar datos) a otras clases.
- **Beneficios:**
 - Mejora la **claridad** y facilita la comprensión del sistema.
 - Hace el código más **reutilizable** y **modular**.
 - Simplifica el **mantenimiento** al evitar clases sobrecargadas.

4. Bajo Acoplamiento

- **Problema:** ¿Cómo reducir dependencias excesivas entre clases?
- **Solución:** Diseñar clases con la menor cantidad de conexiones posibles entre ellas.
- **Definición:**
 - Acoplamiento: medida de fuerza con que un elemento está conectado a otros elementos. Un elemento con bajo acoplamiento no depende de demasiados otros elementos. Una clase con alto (o fuerte) acoplamiento confía en muchas otras clases.
 - La idea es tener la menor cantidad de relaciones posibles entre clases. De tal forma que, en caso de producirse una modificación en alguna de ellas, se tenga la mínima repercusión posible en el resto de las clases
- **Ejemplo:**
 - Si un Pedido necesita calcular el total de productos, debería interactuar solo con clases como Producto, en lugar de acceder a datos innecesarios de otras clases.
- **Beneficios:**
 - Reduce el impacto de cambios en el sistema.
 - Aumenta la **reutilización** de componentes.
 - Facilita el **entendimiento** de las clases por separado.



5. Controlador (el gestor)

- **Problema:** ¿Qué clase debería manejar eventos y coordinar el flujo de datos en un caso de uso?
- **Solución:** Crear una clase controladora que actúe como intermediaria entre la interfaz de usuario y la lógica de negocio. Es la que recibe los datos del usuario y la que los envía a las distintas clases según el método llamado.
- **Tipos de controladores:**
 - **Controlador de fachada:** representa el sistema global o subsistemas completos.
 - **Controlador de casos de uso:** administra un caso de uso específico; no pertenece al dominio del negocio, sino a la lógica de aplicación. Es una clase de fabricación pura o creación arbitraria del diseñador.
- **Ejemplo:**
 - Un controlador de GestiónUsuarios recibe las solicitudes de la interfaz de usuario y las delega a las clases correspondientes (Usuario, BaseDatos).
- **Beneficios:**
 - Centraliza el manejo de la lógica de negocio, facilitando su modificación.
 - Permite reutilizar procesos en diferentes interfaces.
 - Asegura que las operaciones del sistema sean ejecutadas en una secuencia válida.

Beneficios Generales de los Patrones GRASP

1. **Diseño eficiente:** Las responsabilidades son asignadas lógicamente, lo que mejora la claridad y estructura del código.
2. **Mantenimiento y evolución:** El bajo acoplamiento y la alta cohesión facilitan el mantenimiento y futuras actualizaciones.
3. **Colaboración efectiva:** Las clases trabajan en conjunto según sus roles, promoviendo modularidad y adaptabilidad.
4. **Reutilización del código:** Un diseño bien estructurado permite usar componentes en distintos contextos o proyectos.

10. UX (Experiencia de Usuario)

¿Qué hacen los diseñadores de UX?

Los diseñadores de UX crean interfaces útiles y funcionales, asegurando que los usuarios puedan interactuar de manera efectiva con el producto. Se encargan de definir pantallas, servicios, funcionalidades y etiquetas, además de crear prototipos y realizar pruebas para evaluar la experiencia del usuario. Su objetivo es alinear los objetivos del proyecto con las necesidades y expectativas de los usuarios.

Diferencia entre UX y UI

UX (Experiencia de Usuario):

- Se centra en **cómo se siente** el usuario al interactuar con un producto o servicio.

- **Elementos clave:**
 - Arquitectura de la información: Cómo se organiza el contenido.
 - Definición de usuarios y sus necesidades: Investigar y entender qué necesitan los usuarios.
 - Analytics: Medición y análisis de cómo interactúan los usuarios.
 - Wireframes y prototipos: Planificación de flujos y funcionalidad.
 - Diseño interactivo y accesibilidad: Crear interacciones fluidas y usables para todos.
- En resumen: Es **lógica, analítica y práctica**.

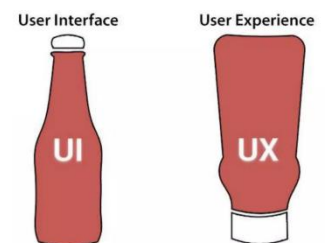
UI (Interfaz de Usuario):

- Se enfoca en **cómo luce y funciona** visualmente el producto.
- **Elementos clave:**
 - Identidad y guía de estilo: La apariencia general que sigue el diseño.
 - Diseño visual: Colores, formas, iconos, etc.
 - Bocetos y tipografía: Representación gráfica del producto.
 - Uso de color: Cómo se aplican colores para resaltar o guiar la interacción.
- En resumen: Es **creativa, emocional y estética**.

Diferencia principal:

- **UX** define la experiencia general y busca resolver problemas para el usuario.
- **UI** es la ejecución visual y estilística de esa experiencia.

- **UX** hace una **interface útil**
- **UI** hace una **interface bella**



IHC → Interacción Humano Computadora

Es una disciplina que se ocupa de estudiar y mejorar la interacción entre las personas y los sistemas computacionales. Su objetivo principal es diseñar, evaluar e implementar sistemas informáticos que sean fáciles de usar, eficientes y satisfactorios para los usuarios.

Disciplinas clave:

1. **Psicología:**
 - Ayuda a entender el comportamiento humano, la percepción y cómo los usuarios toman decisiones al interactuar con sistemas.
2. **Sociología:**
 - Analiza las interacciones humanas en contextos sociales, lo cual influye en el diseño de sistemas colaborativos y sociales.
3. **Diseño:**
 - Cubre la parte estética y funcional, asegurando que los sistemas sean atractivos y agradables de usar.
4. **Ergonomía:**

- Se enfoca en adaptar los sistemas a las capacidades físicas y cognitivas de los usuarios para minimizar el esfuerzo y maximizar la comodidad.

5. Programación:

- Asegura la implementación técnica del diseño y funcionalidad, haciendo posible la interacción entre el usuario y el sistema.

6. Inteligencia Artificial:

- Incorpora sistemas inteligentes que pueden adaptarse a las necesidades del usuario, personalizando y mejorando la experiencia.

7. Ingeniería de Software:

- Proporciona los principios técnicos y metodológicos para desarrollar sistemas robustos, escalables y fáciles de mantener.

Utilidad vs Usabilidad

- **Utilidad:** Se refiere a **qué tan útil** es un sistema o producto para resolver un problema o satisfacer una necesidad. Es decir, si cumple su propósito o función.
- **Usabilidad:** Se enfoca en **qué tan fácil y eficiente** es usar el sistema. Evalúa aspectos como la simplicidad, aprendizaje y satisfacción del usuario al interactuar con él.

Accesibilidad

La **accesibilidad** es la capacidad de un producto para ser utilizado por el mayor número posible de personas, sin importar sus limitaciones físicas o contextuales. Esto implica diseñar interfaces que respondan a las necesidades de todos los usuarios, incluyendo aquellos con discapacidades.

Un producto accesible debe ser:

1. **Perceptible:** La información debe ser accesible para todos los usuarios, incluyendo alternativas para personas con discapacidades visuales o auditivas.
2. **Operable:** El producto debe ser fácil de usar, adaptándose a diferentes dispositivos y necesidades motoras.
3. **Comprensible:** La interfaz debe ser fácil de entender, con interacciones claras y simples.
4. **Robusto:** Debe ser compatible con diversas tecnologías y adaptarse a futuros cambios tecnológicos.

4 elementos de la Empatía

1. Ver su mundo
2. Entender los sentimientos
3. Comunicar entendimientos
4. Apreciarlos como seres humanos

Arquitectura de Información

La **Arquitectura de la Información** es la disciplina que organiza y estructura la información para que los usuarios puedan encontrar y comprender fácilmente lo que buscan.

Puntos clave:

- **Información ≠ Contenido:** La información es el conocimiento procesado, mientras que el contenido es el material bruto. La forma de organizarlo determina su utilidad.
 - **El lenguaje importa:** La organización de los datos debe ser precisa, ya que afecta cómo los usuarios los interpretan.
 - **No hay una sola manera:** La taxonomía (clasificación de información) no es fija, puede variar según el contexto.
 - **Imágenes:** Ayudan a mejorar la comprensión y crear un territorio común.
 - **Arquitectura colaborativa:** La organización de la información requiere colaboración entre diferentes disciplinas.
 - **Siempre existe:** La arquitectura de la información está presente en cualquier sistema que implique organización y navegación de datos.
-

Proceso de UX:

1. Investigar

2. Organizar:

- a. Representar todas las estructuras posibles de los contenidos, en correspondencia a las necesidades de usuarios, propósitos del negocio y contexto de uso.
- b. Hacer corresponder las estructuras planteadas a las necesidades tanto de emisores (clientes) como de receptores (usuarios).
- c. Definir todos los flujos funcionales que tendrá el producto digital, correspondiendo con los flujos reales de los usuarios en su contexto.

3. Creación:

- a. Definir las pantallas del producto (diagramas de presentación o prototipos en baja)
- b. Definir los servicios y funcionalidades que tendrá el producto.
- c. Definir las etiquetas del producto (labeling)
- d. Crear prototipos de alto nivel, funcionales y simulando la interactividad de las tareas a ser testeadas.

4. Evaluación

- a. Pruebas de Prototipos
 - b. Revisión de Diagramas
 - c. Comprobación de Robustez
 - d. Comprensión de los servicios diseñados por parte de los usuarios
-

Componentes de una Experiencia (GARRET)

Capas:

- **Estrategia:** Define el propósito claro de la experiencia, tomando en cuenta tanto las necesidades del usuario como los objetivos del negocio.
- **Alcance:** Establece las especificaciones funcionales y los requerimientos de contenido, priorizando características y funciones. Ejemplo: en un reproductor de música, la funcionalidad sería pausar y adelantar, mientras que el contenido son los archivos musicales.
- **Estructura:**

- **Arquitectura de Información:** Cómo se organiza la información.
- **Diseño de Interacción:** El flujo de usuario, es decir, cómo el producto le ayuda a cumplir sus necesidades.
- **Esqueleto:**
 - Diseño de la interfaz y la información.
 - Creación de **Wireframes**, que especifican:
 - Qué se muestra en pantalla.
 - Cómo y dónde están los elementos interactivos.
 - La navegación entre contenidos.
 - Cómo se facilita el cumplimiento de las tareas.
- **Contenido Visual:**
 - Enfoque en la **experiencia sensorial** y el diseño visual
 - Última capa en la que se definen componentes visuales como colores, tipografías y otros detalles de diseño para asegurar que los elementos sean legibles y coherentes con la experiencia.

Brechas de Ejecución y Evaluación en la Interacción

1. **Brecha de Ejecución:**
Ocurre cuando el usuario no puede relacionar claramente sus intenciones con las acciones disponibles en el sistema. Es decir, el usuario no sabe cómo interactuar con el sistema para lograr lo que desea debido a una falta de claridad en las opciones que el sistema le ofrece.
2. **Brecha de Evaluación:**
Sucedre cuando el usuario no puede interpretar correctamente la respuesta del sistema, o cuando la respuesta que recibe no corresponde con lo que esperaba. Esta brecha implica una desconexión entre la acción del usuario y la retroalimentación proporcionada por el sistema, dificultando la evaluación de si la acción fue exitosa o no.

Heurísticas como Derechos

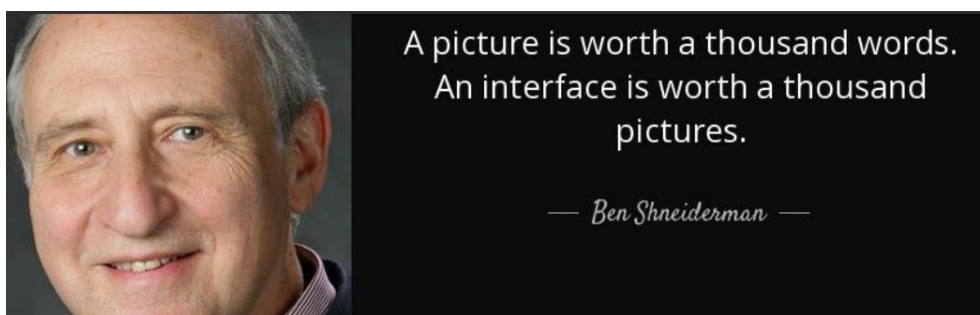
- **Entendimiento:** El sistema debe ser comprensible y fácil de navegar, con un lenguaje claro.
- **Operación:** El sistema debe ser fácil de usar, permitiendo al usuario interactuar de manera intuitiva.
- **Retroalimentación:** El sistema debe proporcionar respuestas claras sobre las acciones del usuario para asegurar que sabe si ha sido exitoso o necesita ajustar algo.



Las **8 reglas de Schneiderman** son principios de diseño de interfaces de usuario que mejoran la usabilidad:

1. **Buscar siempre la coherencia:** Mantén un diseño consistente en todo el sistema (que la cruz este arriba a la derecha)
2. **Permitir el uso de 'short-cuts':** Facilita el acceso rápido a funciones comunes. (alt f4)

3. **Dar realimentación de información:** Para cada acción del operador, debe haber una respuesta del sistema.
4. **Diseñar diálogos que tengan un fin:** Cada interacción debe tener un propósito claro.
5. **Permitir manejos simples de los errores:** Ofrece soluciones claras y fáciles cuando ocurran errores
6. **Permitir deshacer las acciones con facilidad:** Deja que el usuario revierta acciones fácilmente (ctrl z)
7. **Permitir que el centro de control sea interno:** El usuario debe sentir que controla la interacción.
8. **Reducir la carga de la memoria inmediata:** Evita que el usuario tenga que recordar mucha información, mostrando datos relevantes en la pantalla.



"Una imagen vale más que mil palabras. Una interfaz vale más que mil imágenes"

Una imagen puede transmitir mucha información de forma visual y rápida, pero una interfaz bien diseñada permite no solo interpretar información, sino también interactuar de manera intuitiva y eficiente con un sistema complejo. En este contexto, la interfaz actúa como un puente que simplifica tareas, optimiza la experiencia del usuario y hace que las funciones de un sistema sean accesibles y comprensibles, superando incluso el impacto de las imágenes al proporcionar interacción dinámica y práctica

Algunas Herramientas

- **MoodBoard (tablero de humor):** Collage visual que inspira el diseño del producto, estableciendo una estética o estilo general.
- **Mapa de Empatía:** Herramienta para entender a los usuarios, explorando qué piensan, sienten, ven y hacen.
- **User Journey:** Representación de cómo un usuario interactúa con el producto, identificando puntos clave y problemas.
- **Wireframes:** Esquema básico de la estructura del producto. Puede ser:

- **Baja tecnología:** Hecho en papel o con herramientas simples. **Media tecnología:** Usando software con más detalles.
- **Alta tecnología:** Prototipo casi funcional, más detallado.

- **Mock-ups:** Diseño estático de fidelidad media-alta que muestra cómo debería lucir el producto final.
- **Prototipos:** Representación interactiva y funcional del producto, casi idéntico al diseño final.
- **Diagrama de navegabilidad:** Esquema que muestra cómo se conectan las diferentes secciones o pantallas del producto.



• Nivel de Especialización

- Un diseñador UX ideal comienza como generalista, entendiendo el proceso general de diseño, herramientas y metodologías.

- Puede especializarse en áreas específicas (como investigación, diseño visual, arquitectura de información), aunque esto requiere tiempo y profundización.
- Hay roles tanto para generalistas (que coordinan equipos) como para especialistas (enfocados en áreas específicas).

- **Entendiendo Roles y Colaboración**

- Cada especialista en UX debe comprender cómo su trabajo impacta y es interpretado por otros roles (investigador, arquitecto, diseñador visual, etc.).
- Es clave conocer las responsabilidades propias y de otros, colaborando para un entregable final que satisfaga las necesidades del usuario.

- **Profundización de Conocimientos**

- No basta con años de experiencia en una tarea; se requiere ampliar y profundizar conocimientos en las áreas propias y generales.
- El diseño UX no es lineal, sino un "mapa de calor" donde cada profesional enfoca su desarrollo en distintas áreas según sus intereses y habilidades.

